

Python Advanced Concepts

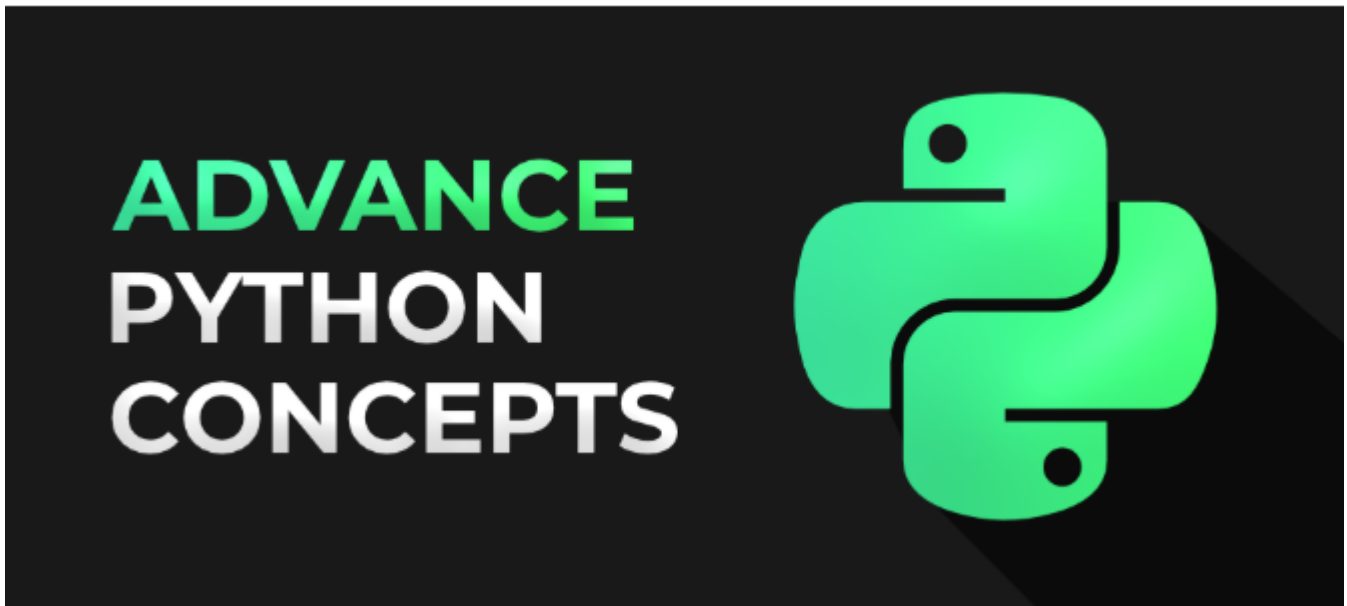


Table of Contents

1. [Introduction to Python](#)
2. [Object-Oriented Programming](#)
3. [Data Structures](#)
4. [Decorators](#)
5. [Generators](#)
6. [Async Programming](#)
7. [Logging](#)
8. [Configuration Management](#)
9. [Python Best Practices](#)
10. [Conclusion](#)
11. [References](#)

Introduction to Advanced Python Concepts

- [What are Advanced Python Concepts](#)
- [Why Learn Advanced Python Concepts](#)
- [Applications of Advanced Python Concepts](#)

What are Advanced Python Concepts

Advanced Python Concepts are powerful features and programming techniques that help developers write efficient, scalable, maintainable, and professional Python applications.

These concepts go beyond basic Python programming and focus on improving code organization, performance, reusability, and application design.

Advanced Python features are widely used in real-world software development, web applications, automation tools, cloud services, data science projects, and machine learning systems.

Why Learn Advanced Python Concepts

Learning advanced concepts helps developers:

- Write cleaner and reusable code
- Improve application performance
- Handle large datasets efficiently
- Build scalable applications
- Follow industry-standard development practices
- Improve debugging and maintainability
- Prepare for professional software development

Applications of Advanced Python Concepts

Advanced Python concepts are used in:

- Web Development
- Data Science
- Machine Learning
- Artificial Intelligence
- Cloud Computing

Object-Oriented Programming

- **Overview**
 - **Class**
 - **Object**
 - **Class and Object**
 - **Inheritance**
 - **Encapsulation**
 - **Polymorphism**
 - **Abstraction**
 - **Multiple Inheritance**
 - **Method Overriding**
 - **Operator Overloading**
 - **Composition**
 - **Summary**
-

Object-Oriented Programming (OOP)

Overview

Object-Oriented Programming (OOP) is a programming paradigm that uses classes and objects to organize code. It helps create modular, reusable, and maintainable applications.

Why OOP?

- Organizes code into classes and objects
- Improves code reusability
- Makes applications easier to maintain
- Supports real-world modeling
- Enhances scalability

Detailed Examples

1. Class

 Class Diagram

Explanation

A class is a blueprint for creating objects. It defines the attributes and methods that objects can have.

Example: A Vehicle class acts as a blueprint for creating different vehicle objects such as a Bicycle, Car, and Scooter.

2. Object

 Object Diagram

Explanation

An object is an instance of a class. Each object contains its own state and behavior.

Example: A Bike object can have an identity (Bike ID), state (color, model), and behavior (drive, brake, accelerate).

3. Class and object

Syntax

```
class ClassName:

    def __init__(self, attribute1, attribute2):
        self.attribute1 = attribute1
        self.attribute2 = attribute2

    def method_name(self):
        print("Action")

object_name = ClassName(value1, value2)

object_name.method_name()
```

Syntax Explanation

- `class` is used to create a class.
- `__init__()` is the constructor that initializes object data.
- `self` refers to the current object.
- Methods define the actions an object can perform.
- `ClassName()` creates an object of the class.
- Methods are called using the object.

Code

```
class Vehicle:
    def __init__(self, identity, color):
        self.identity = identity
        self.color = color

    def drive(self):
        print(f"{self.identity} is driving")

    def brake(self):
        print(f"{self.identity} is braking")

    def accelerate(self):
        print(f"{self.identity} is accelerating")

# Creating an object
bike = Vehicle("Bike", "Red")

print("Identity:", bike.identity)
print("Color:", bike.color)

bike.drive()
```

```
bike.brake()  
bike.accelerate()
```

Code Explanation

Step 1: Define the Class

```
class Vehicle:
```

Defines a class named `Vehicle`. A class acts as a blueprint for creating objects.

Step 2: Create the Constructor

```
def __init__(self, identity, color):  
    self.identity = identity  
    self.color = color
```

The `__init__()` constructor initializes the object's data.

- `identity` stores the type of vehicle.
 - `color` stores the color of the vehicle.
 - `self` refers to the current object being created.
-

Step 3: Define the Methods

```
def drive(self):  
    print(f"{self.identity} is driving")
```

Defines the `drive()` method, which displays a message indicating that the vehicle is driving.

```
def brake(self):  
    print(f"{self.identity} is braking")
```

Defines the `brake()` method, which displays a message indicating that the vehicle is braking.

```
def accelerate(self):  
    print(f"{self.identity} is accelerating")
```

Defines the `accelerate()` method, which displays a message indicating that the vehicle is accelerating.

Step 4: Create an Object

```
bike = Vehicle("Bike", "Red")
```

Creates an object named `bike` from the `Vehicle` class.

The constructor receives:

- `"Bike"` as the vehicle identity.
- `"Red"` as the vehicle color.

Step 5: Access Object Attributes

```
print("Identity:", bike.identity)
print("Color:", bike.color)
```

Accesses and displays the object's attributes using the dot (`.`) operator.

Step 6: Call the Methods

```
bike.drive()
bike.brake()
bike.accelerate()
```

Calls the methods of the `bike` object.

Each method performs its corresponding action and displays the output.

Output

```
Identity: Bike
Color: Red
Bike is driving
Bike is braking
Bike is accelerating
```



Class and Object Output

Key Takeaway

- A **Class** is a blueprint for creating objects.
- An **Object** is an instance of a class.
- The `__init__()` constructor initializes object data.
- Attributes store the object's data.
- Methods define the object's behavior.
- Objects access attributes and methods using the dot (.) operator.

4. Inheritance

Diagram

 Inheritance Diagram

Explanation

Inheritance allows a child class to inherit properties and methods from a parent class.

Syntax

```
class ParentClass:
    # Parent class members

class ChildClass(ParentClass):
    # Child class members
```

Syntax Explanation

- `class ParentClass:` defines the parent (base) class.
 - `class ChildClass(ParentClass):` defines the child (derived) class.
 - The child class inherits the properties and methods of the parent class.
-

Example

Example: A Student class can inherit common properties and methods, such as the name, from a Person class while adding its own property, such as the course.

Code

02_inheritance.py

```
# Parent Class
class Person:
    def __init__(self, name):
```

```
        self.name = name

    def display(self):
        print("Name:", self.name)

# Child Class
class Student(Person):
    def __init__(self, name, course):
        super().__init__(name)
        self.course = course

    def student_details(self):
        self.display()
        print("Course:", self.course)

# Creating Object
student = Student("Avinash", "Artificial Intelligence")

print("Student Details")
student.student_details()
```

Code Explanation

Step 1: Define the Parent Class

```
class Person:
```

Defines the parent (base) class named **Person**.

The parent class contains common properties and methods that can be inherited by child classes.

Step 2: Create the Parent Constructor

```
def __init__(self, name):
    self.name = name
```

The constructor initializes the **name** attribute of the **Person** class.

This attribute will be inherited by the child class.

Step 3: Define the Parent Method

```
def display(self):  
    print("Name:", self.name)
```

Defines the `display()` method, which prints the person's name.

This method is inherited by the child class.

Step 4: Define the Child Class

```
class Student(Person):
```

Creates a child class named `Student` that inherits from the `Person` class.

The child class can access the properties and methods of the parent class.

Step 5: Call the Parent Constructor

```
super().__init__(name)
```

Calls the constructor of the parent class to initialize the inherited `name` attribute.

Using `super()` avoids rewriting the same initialization code.

Step 6: Add a Child-Specific Attribute

```
self.course = course
```

Initializes the `course` attribute, which is specific to the `Student` class.

Step 7: Define the Child Method

```
def student_details(self):  
    self.display()  
    print("Course:", self.course)
```

Defines a method that:

- Calls the inherited `display()` method to print the student's name.
- Prints the student's course.

Step 8: Create an Object

```
student = Student("Avinash", "Artificial Intelligence")
```

Creates an object named `student` from the `Student` class.

The constructor receives:

- `"Avinash"` as the name.
- `"Artificial Intelligence"` as the course.

Step 9: Call the Child Method

```
student.student_details()
```

Calls the `student_details()` method.

This method displays both the inherited information (`name`) and the child-specific information (`course`).

Output

 Inheritance Output

Key Takeaway

- **Inheritance** allows one class to inherit the properties and methods of another class.
- The **Parent Class** contains common features.
- The **Child Class** reuses the parent class features and can add its own.
- `super()` is used to call the parent class constructor.
- Inheritance promotes **code reuse**, **maintainability**, and **organized object-oriented design**.

5. Encapsulation

Diagram

 Encapsulation Diagram

Explanation

Encapsulation protects data by restricting direct access and providing controlled access through methods.

Syntax

```

class ClassName:

    def __init__(self):
        self.public_attribute = value
        self.__private_attribute = value

    def public_method(self):
        pass

    def get_private_attribute(self):
        return self.__private_attribute

# Creating Object
object_name = ClassName()

object_name.public_method()
print(object_name.get_private_attribute())

```

Syntax Explanation

- `class ClassName:` defines a class.
- `self.public_attribute` is a **public attribute** that can be accessed directly.
- `self.__private_attribute` is a **private attribute** that cannot be accessed directly outside the class.
- Public methods are used to access or modify private attributes.
- `ClassName()` creates an object of the class.
- Methods are called using the object.

Example

Example: A BankAccount class protects account balances from unauthorized modification.

Code

03_encapsulation.py

```

class BankAccount:
    def __init__(self, account_holder, balance):
        self.account_holder = account_holder
        self.__balance = balance # Private attribute

    def deposit(self, amount):
        self.__balance += amount
        print(f"Deposited: ₹{amount}")

    def withdraw(self, amount):
        if amount <= self.__balance:

```

```
        self.__balance -= amount
        print(f"Withdrawn: ₹{amount}")
    else:
        print("Insufficient Balance")

    def get_balance(self):
        return self.__balance

account = BankAccount("Avinash", 10000)

account.deposit(2000)
account.withdraw(1500)

print("Current Balance:", account.get_balance())
```

Code Explanation

Step 1: Define the Class

```
class BankAccount:
```

Defines a class named `BankAccount`.

This class represents a bank account and contains the account holder's information and balance.

Step 2: Create the Constructor

```
def __init__(self, account_holder, balance):
    self.account_holder = account_holder
    self.__balance = balance
```

The constructor initializes the object's data.

- `account_holder` stores the account holder's name.
 - `__balance` stores the account balance as a **private attribute**.
 - The double underscore (`__`) makes the balance inaccessible directly from outside the class.
-

Step 3: Define the Deposit Method

```
def deposit(self, amount):
    self.__balance += amount
    print(f"Deposited: ₹{amount}")
```

The `deposit()` method adds the specified amount to the private balance and displays a confirmation message.

Step 4: Define the Withdraw Method

```
def withdraw(self, amount):
    if amount <= self.__balance:
        self.__balance -= amount
        print(f"Withdrawn: ₹{amount}")
    else:
        print("Insufficient Balance")
```

The `withdraw()` method checks whether sufficient balance is available.

- If enough balance exists, the amount is deducted.
 - Otherwise, an **"Insufficient Balance"** message is displayed.
-

Step 5: Define the Getter Method

```
def get_balance(self):
    return self.__balance
```

The `get_balance()` method returns the current account balance.

Since the balance is private, it can only be accessed through this public method.

Step 6: Create an Object

```
account = BankAccount("Avinash", 10000)
```

Creates an object named `account`.

The constructor initializes:

- Account Holder → "Avinash"
 - Initial Balance → 10000
-

Step 7: Deposit Money

```
account.deposit(2000)
```

Deposits ₹2000 into the account.

Updated Balance:

```
10000 + 2000 = 12000
```

Step 8: Withdraw Money

```
account.withdraw(1500)
```

Withdraws ₹1500 from the account.

Updated Balance:

```
12000 - 1500 = 10500
```

Step 9: Display the Current Balance

```
print("Current Balance:", account.get_balance())
```

Calls the `get_balance()` method to retrieve and display the current balance.

Output

 Encapsulation Output

Key Takeaway

- **Encapsulation** protects an object's data by restricting direct access.
 - **Private attributes** are created using a double underscore (`__`).
 - **Public methods** provide controlled access to private data.
 - Encapsulation improves **security**, **data integrity**, and **maintainability**.
-

6. Polymorphism

Diagram



Explanation

Polymorphism allows the same method to behave differently for different objects.

Syntax

```
class Class1:
    def method_name(self):
        pass

class Class2:
    def method_name(self):
        pass

objects = [Class1(), Class2()]

for obj in objects:
    obj.method_name()
```

Syntax Explanation

- Different classes define a method with the **same name**.
- Each class provides its own implementation of that method.
- Objects of different classes are stored in a collection.
- Calling the same method on each object executes the appropriate implementation.
- This behavior is known as **Polymorphism**.

Example

Example: Different vehicles can implement the same start() method differently.

Code

04_polymorphism.py

```
class Car:
    def start(self):
        print("Car starts with a key or push button")
```

```
class Bike:
    def start(self):
        print("Bike starts with a self-start button")

class Truck:
    def start(self):
        print("Truck starts with a heavy-duty ignition system")

vehicles = [Car(), Bike(), Truck()]

for vehicle in vehicles:
    vehicle.start()
```

Code Explanation

Step 1: Define the Car Class

```
class Car:
```

Defines a class named `Car`.

The `Car` class provides its own implementation of the `start()` method.

Step 2: Define the start() Method for Car

```
def start(self):
    print("Car starts with a key or push button")
```

When the `start()` method is called on a `Car` object, it displays how a car starts.

Step 3: Define the Bike Class

```
class Bike:
```

Defines a class named `Bike`.

Like the `Car` class, it also contains a `start()` method.

Step 4: Define the start() Method for Bike

```
def start(self):  
    print("Bike starts with a self-start button")
```

When the `start()` method is called on a `Bike` object, it displays how a bike starts.

Step 5: Define the Truck Class

```
class Truck:
```

Defines a class named `Truck`.

This class also implements the `start()` method.

Step 6: Define the start() Method for Truck

```
def start(self):  
    print("Truck starts with a heavy-duty ignition system")
```

When the `start()` method is called on a `Truck` object, it displays how a truck starts.

Step 7: Create Objects

```
vehicles = [Car(), Bike(), Truck()]
```

Creates objects of the `Car`, `Bike`, and `Truck` classes and stores them in a list.

Step 8: Call the Same Method

```
for vehicle in vehicles:  
    vehicle.start()
```

The loop calls the same `start()` method for every object.

Although the method name is the same, each object executes its own implementation.

This demonstrates **Polymorphism**, where the same method behaves differently for different objects.

Output

Key Takeaway

- **Polymorphism** allows different classes to define the **same method** in different ways.
 - The same method call can produce different behavior depending on the object.
 - It improves **code flexibility**, **reusability**, and **maintainability**.
 - Polymorphism is one of the four fundamental principles of Object-Oriented Programming (OOP).
-

7. Abstraction

Diagram



Explanation

Abstraction hides implementation details and exposes only essential functionality.

Syntax

```
from abc import ABC, abstractmethod

class AbstractClass(ABC):

    @abstractmethod
    def method_name(self):
        pass

class ChildClass(AbstractClass):

    def method_name(self):
        # Implementation
        pass

object_name = ChildClass()
object_name.method_name()
```

Syntax Explanation

- **ABC** is used to create an abstract base class.
- **@abstractmethod** declares a method that must be implemented by child classes.
- The child class inherits from the abstract class.

- The child class provides the implementation of the abstract method.
- An object is created only for the child class, not the abstract class.

Example

Example: Users drive a car without knowing the internal engine implementation.

Code

05_abstraction.py

```
from abc import ABC, abstractmethod

class Vehicle(ABC):

    @abstractmethod
    def start(self):
        pass

class Car(Vehicle):

    def start(self):
        print("Car engine started")

car = Car()
car.start()
```

Code Explanation

Step 1: Import the Required Module

```
from abc import ABC, abstractmethod
```

Imports the `ABC` (Abstract Base Class) and `abstractmethod` from Python's `abc` module.

These are used to create abstract classes and abstract methods.

Step 2: Define the Abstract Class

```
class Vehicle(ABC):
```

Defines an abstract class named `Vehicle`.

An abstract class cannot be instantiated directly. It serves as a blueprint for its child classes.

Step 3: Declare an Abstract Method

```
@abstractmethod  
def start(self):  
    pass
```

Declares the `start()` method as an abstract method.

Every child class that inherits from `Vehicle` **must** provide its own implementation of this method.

Step 4: Define the Child Class

```
class Car(Vehicle):
```

Creates a child class named `Car` that inherits from the abstract class `Vehicle`.

Step 5: Implement the Abstract Method

```
def start(self):  
    print("Car engine started")
```

Provides the implementation of the abstract `start()` method.

Since the method is implemented, the `Car` class becomes a concrete class and objects can be created from it.

Step 6: Create an Object

```
car = Car()
```

Creates an object named `car` from the `Car` class.

Unlike the abstract class, the child class can be instantiated because it implements all abstract methods.

Step 7: Call the Method

```
car.start()
```

Calls the `start()` method of the `Car` object.

The implementation defined in the `Car` class is executed.

Output

 Abstraction Output

Key Takeaway

- **Abstraction** hides implementation details and exposes only the essential functionality.
 - An **Abstract Class** acts as a blueprint for other classes.
 - **Abstract Methods** must be implemented by every child class.
 - Objects cannot be created from an abstract class.
 - Abstraction improves **security**, **code organization**, and **maintainability** by focusing on **what** an object does rather than **how** it does it.
-

8. Multiple Inheritance

Diagram

 Multiple Inheritance Diagram

Explanation

Multiple inheritance allows a class to inherit from more than one parent class.

Syntax

```
class ParentClass1:
    pass

class ParentClass2:
    pass

class ChildClass(ParentClass1, ParentClass2):
    pass
```

```
object_name = ChildClass()
```

Syntax Explanation

- `class ParentClass1:` defines the first parent class.
- `class ParentClass2:` defines the second parent class.
- `class ChildClass(ParentClass1, ParentClass2):` creates a child class that inherits from both parent classes.
- The child class can access the properties and methods of both parent classes.
- `ChildClass()` creates an object of the child class.

Example

Example: A Child class can inherit skills from both the Father class and the Mother class. The child can access methods from both parent classes through multiple inheritance.

Code

06_multiple_inheritance.py

```
class Father:
    def skill1(self):
        print("Programming")

class Mother:
    def skill2(self):
        print("Teaching")

class Child(Father, Mother):
    pass

c = Child()

c.skill1()
c.skill2()
```

Code Explanation

Step 1: Define the First Parent Class

```
class Father:
```

Defines the first parent class named `Father`.

This class contains the `skill1()` method.

Step 2: Define the skill1() Method

```
def skill1(self):  
    print("Programming")
```

Defines the `skill1()` method, which displays the programming skill inherited from the `Father` class.

Step 3: Define the Second Parent Class

```
class Mother:
```

Defines the second parent class named `Mother`.

This class contains the `skill2()` method.

Step 4: Define the skill2() Method

```
def skill2(self):  
    print("Teaching")
```

Defines the `skill2()` method, which displays the teaching skill inherited from the `Mother` class.

Step 5: Define the Child Class

```
class Child(Father, Mother):  
    pass
```

Creates a child class named `Child` that inherits from both the `Father` and `Mother` classes.

As a result, the child class can access the methods of both parent classes.

Step 6: Create an Object

```
c = Child()
```

Creates an object named `c` from the `Child` class.

Step 7: Call the Inherited Methods

```
c.skill1()  
c.skill2()
```

Calls the methods inherited from both parent classes.

- `skill1()` is inherited from the `Father` class.
- `skill2()` is inherited from the `Mother` class.

This demonstrates **Multiple Inheritance**, where a single child class inherits features from multiple parent classes.

Output

 Multiple Inheritance Output

Key Takeaway

- **Multiple Inheritance** allows a child class to inherit from **more than one parent class**.
- The child class can access the properties and methods of all parent classes.
- It promotes **code reuse** by combining functionality from multiple classes.
- Multiple inheritance is useful when a class needs features from different parent classes.

9. Method Overriding

Diagram

 Method Overriding Diagram

Explanation

Method overriding allows a child class to provide its own implementation of a parent method.

Syntax

```
class ParentClass:  
  
    def method_name(self):  
        pass  
  
class ChildClass(ParentClass):
```

```
def method_name(self):
    # Overridden implementation
    pass

object_name = ChildClass()
object_name.method_name()
```

Syntax Explanation

- `class ParentClass:` defines the parent class.
- `class ChildClass(ParentClass):` creates a child class that inherits from the parent class.
- The child class defines a method with the **same name** as the parent class.
- The child class method **overrides** the parent class method.
- When the method is called using the child object, the child class implementation is executed.

Example

Example: Different vehicle types can provide their own implementation of mileage and cost calculations.

Code

07_method_overriding.py

```
class Vehicle:

    def get_mileage(self):
        print("Mileage information not available")

    def get_cost(self):
        print("Cost information not available")

class Car(Vehicle):

    def get_mileage(self):
        print("Car Mileage: 18 km/l")

    def get_cost(self):
        print("Car Cost: ₹10,00,000")

class Bike(Vehicle):

    def get_mileage(self):
        print("Bike Mileage: 55 km/l")
```

```
def get_cost(self):  
    print("Bike Cost: ₹1,20,000")  
  
car = Car()  
bike = Bike()  
  
print("Car Details:")  
car.get_mileage()  
car.get_cost()  
  
print("\nBike Details:")  
bike.get_mileage()  
bike.get_cost()
```

Code Explanation

Step 1: Define the Parent Class

```
class Vehicle:
```

Defines a parent class named `Vehicle`.

This class provides general methods that can be inherited by child classes.

Step 2: Define the Parent Methods

```
def get_mileage(self):  
    print("Mileage information not available")
```

Defines the `get_mileage()` method, which displays a default mileage message.

```
def get_cost(self):  
    print("Cost information not available")
```

Defines the `get_cost()` method, which displays a default cost message.

These methods can be overridden by child classes.

Step 3: Define the Car Class

```
class Car(Vehicle):
```

Creates a child class named **Car** that inherits from the **Vehicle** class.

Step 4: Override the Parent Methods in Car

```
def get_mileage(self):  
    print("Car Mileage: 18 km/l")
```

Overrides the parent class `get_mileage()` method with car-specific mileage information.

```
def get_cost(self):  
    print("Car Cost: ₹10,00,000")
```

Overrides the parent class `get_cost()` method with car-specific cost information.

Step 5: Define the Bike Class

```
class Bike(Vehicle):
```

Creates another child class named **Bike** that also inherits from the **Vehicle** class.

Step 6: Override the Parent Methods in Bike

```
def get_mileage(self):  
    print("Bike Mileage: 55 km/l")
```

Overrides the parent class `get_mileage()` method with bike-specific mileage information.

```
def get_cost(self):  
    print("Bike Cost: ₹1,20,000")
```

Overrides the parent class `get_cost()` method with bike-specific cost information.

Step 7: Create Objects

```
car = Car()
bike = Bike()
```

Creates two objects:

- `car` from the `Car` class.
 - `bike` from the `Bike` class.
-

Step 8: Call the Overridden Methods

```
print("Car Details:")
car.get_mileage()
car.get_cost()
```

Calls the overridden methods of the `Car` class.

Since these methods override the parent methods, the car-specific information is displayed.

Step 9: Call the Bike Methods

```
print("\nBike Details:")
bike.get_mileage()
bike.get_cost()
```

Calls the overridden methods of the `Bike` class.

The bike-specific implementations are executed instead of the parent class methods.

This demonstrates **Method Overriding**, where child classes provide their own implementation of methods defined in the parent class.

Output

 Method Overriding Output

Key Takeaway

- **Method Overriding** allows a child class to provide its own implementation of a method inherited from the parent class.
- The child class method must have the **same name** as the parent class method.
- When a method is called using a child object, the **child class implementation** is executed instead of the parent class implementation.

- Method overriding enables **runtime polymorphism**, improves **flexibility**, and allows different classes to provide behavior specific to their needs.

10. Operator Overloading

Diagram

Operator Overloading Diagram

Explanation

Operator overloading allows operators to work with user-defined objects.

Syntax

```
class ClassName:

    def __init__(self):
        pass

    def __operator__(self, other):
        # Custom implementation
        pass

object1 = ClassName()
object2 = ClassName()

result = object1 + object2
```

Syntax Explanation

- `__init__()` initializes the object.
- `__operator__()` is a special (magic) method used to overload an operator.
- `other` represents the second object involved in the operation.
- Python automatically calls the corresponding magic method when an operator is used.
- The result of the operation is returned.

Example

Example: Adding two Point objects using the + operator.

Code

`08_operator_overloading.py`

```
class Point:

    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Point(
            self.x + other.x,
            self.y + other.y
        )

    def __str__(self):
        return f"({self.x}, {self.y})"

p1 = Point(2, 3)
p2 = Point(4, 5)

print(p1 + p2)
```

Code Explanation

Step 1: Define the Class

```
class Point:
```

Defines a class named `Point`.

Each object of this class represents a point with **x** and **y** coordinates.

Step 2: Create the Constructor

```
def __init__(self, x, y):
    self.x = x
    self.y = y
```

The constructor initializes the object's data.

- `x` stores the x-coordinate.
 - `y` stores the y-coordinate.
-

Step 3: Overload the + Operator

```
def __add__(self, other):
```

Defines the special method `__add__()`.

Python automatically calls this method whenever the `+` operator is used between two `Point` objects.

The parameter `other` represents the second object.

Step 4: Add the Coordinates

```
return Point(  
    self.x + other.x,  
    self.y + other.y  
)
```

Creates and returns a new `Point` object.

The new object's coordinates are calculated by adding the corresponding coordinates of both objects.

For example:

```
Point 1 = (2, 3)  
Point 2 = (4, 5)  
  
New Point = (6, 8)
```

Step 5: Define the String Representation

```
def __str__(self):  
    return f"({self.x}, {self.y})"
```

Defines how the object should be displayed when passed to the `print()` function.

Instead of displaying the object's memory address, it displays the coordinates in a readable format.

Example:

```
(6, 8)
```

Step 6: Create Objects

```
p1 = Point(2, 3)
p2 = Point(4, 5)
```

Creates two `Point` objects.

- `p1` has coordinates `(2, 3)`.
- `p2` has coordinates `(4, 5)`.

Step 7: Use the + Operator

```
print(p1 + p2)
```

Uses the `+` operator to add the two `Point` objects.

Python automatically calls the `__add__()` method.

The returned object is displayed using the `__str__()` method.

Output:

```
(6, 8)
```

This demonstrates **Operator Overloading**, where the `+` operator is given a custom behavior for user-defined objects.

Output

Operator Overloading Output

Key Takeaway

- **Operator Overloading** allows built-in operators to work with user-defined objects.
- Special methods such as `__add__()` define how operators behave.
- Python automatically calls these special methods when operators are used.
- Operator overloading makes objects easier to use and improves code readability.

11. Composition

Diagram

Composition Diagram

Explanation

Composition represents a HAS-A relationship, where one class contains an object of another class as one of its members.

The contained object works as a part of the main object and is used to perform specific tasks.

Syntax

```
class Component:
    pass

class Container:

    def __init__(self):
        self.component = Component()

object_name = Container()
```

Syntax Explanation

- `class Component`: defines the class that will be used inside another class.
- `class Container`: defines the class that contains the component.
- `self.component = Component()` creates an object of the component class inside the container class.
- This represents a **HAS-A** relationship.
- `Container()` creates an object of the container class.

Example

Example: A Car HAS-A Engine. The Engine object exists inside the Car object.

Code

09_composition.py

```
class Engine:
    def start(self):
        print("Engine Started")

class Car:
    def __init__(self):
        self.engine = Engine()
```

```
def start_car(self):  
    print("Starting Car...")  
    self.engine.start()
```

```
car = Car()  
car.start_car()
```

Code Explanation

Step 1: Define the Engine Class

```
class Engine:
```

Defines a class named **Engine**.

This class represents the engine of a car and contains the functionality related to starting the engine.

Step 2: Define the Engine Method

```
def start(self):  
    print("Engine Started")
```

Defines the **start()** method.

When this method is called, it displays the message "**Engine Started**".

Step 3: Define the Car Class

```
class Car:
```

Defines a class named **Car**.

The **Car** class contains an **Engine** object, demonstrating a **HAS-A** relationship.

Step 4: Create an Engine Object Inside the Car

```
def __init__(self):  
    self.engine = Engine()
```

The constructor creates an object of the `Engine` class and stores it in the `engine` attribute.

This means every `Car` object automatically has its own `Engine` object.

This is an example of **Composition**.

Step 5: Define the `start_car()` Method

```
def start_car(self):  
    print("Starting Car...")  
    self.engine.start()
```

Defines the `start_car()` method.

It performs two actions:

- Displays "**Starting Car...**"
- Calls the `start()` method of the `Engine` object using:

```
self.engine.start()
```

This shows how one object uses another object's functionality.

Step 6: Create a Car Object

```
car = Car()
```

Creates an object named `car` from the `Car` class.

During object creation, the constructor automatically creates an `Engine` object.

Step 7: Call the `start_car()` Method

```
car.start_car()
```

Calls the `start_car()` method of the `Car` object.

The output is produced in the following order:

1. The car starts.
2. The engine starts.

This demonstrates **Composition**, where the **Car** object uses its own **Engine** object to perform a task.

Output

 Composition Output

Key Takeaway

- **Composition** represents a **HAS-A** relationship between classes.
 - One class contains an object of another class as one of its members.
 - The contained object performs specific tasks for the main object.
 - Composition promotes **code reuse**, **modularity**, and **better software design**.
 - A real-world example is a **Car HAS-A Engine**, where the car uses the engine to perform its operations.
-

Summary

Object-Oriented Programming (OOP) helps developers build modular, reusable, and maintainable software.

The key concepts covered in this document are:

- Class
- Object
- Inheritance
- Encapsulation
- Polymorphism
- Abstraction
- Multiple Inheritance
- Method Overriding
- Operator Overloading
- Composition

These concepts form the foundation of object-oriented software design and are widely used in real-world applications.

Data Structures

- **Overview**
- **Lists**
- **Tuples**
- **Sets**

- **Dictionaries**
- **Stack**
- **Queue**
- **Linked List**
- **Deque**
- **Priority Queue**
- **Hash Table**
- **Learning Outcome**
- **Summary**

Data Structures Overview

Data Structures are specialized ways of organizing, storing, and managing data in a computer so that it can be accessed and modified efficiently. Choosing the right data structure is important because it directly affects the performance, memory usage, and scalability of an application.

In Python, several built-in data structures such as Lists, Tuples, Sets, and Dictionaries are available for handling different types of data. More advanced data structures such as Stacks, Queues, Linked Lists, Priority Queues, and Hash Tables can also be implemented to solve complex real-world problems.

Data Structures form the foundation of computer science and software development. Almost every application, from social media platforms and search engines to banking systems and operating systems, relies on data structures for efficient data processing.

Why Data Structures?

- Organize data efficiently
- Improve application performance
- Reduce processing time
- Optimize memory usage
- Support faster searching and sorting
- Enable scalable software design
- Simplify problem solving

Real-World Applications

Data Structures are used in many real-world systems:

- Browser history uses Stacks
- Ticket reservation systems use Queues
- Contact directories use Hash Tables
- GPS navigation systems use Graphs
- Music playlists use Linked Lists
- Hospital emergency systems use Priority Queues
- Social media applications use Trees and Graphs

Concepts Covered

This module covers the following Data Structures:

- Lists
- Tuples
- Sets
- Dictionaries
- Stacks
- Queues
- Linked Lists
- Deques
- Priority Queues
- Hash Tables

1. Lists

Diagram



Explanation

A List is one of the most commonly used data structures in Python. It is used to store an ordered collection of items in a single variable.

Lists are dynamic, meaning elements can be added, updated, or removed after creation. They can store multiple data types such as integers, strings, floating-point numbers, and even other lists.

Characteristics of Lists

- Ordered collection of elements
- Mutable (can be modified after creation)
- Allows duplicate values
- Supports indexing
- Can store multiple data types

Syntax

```
list_name = [item1, item2, item3]

list_name.append(item)
list_name[index] = new_value
list_name.remove(item)

for item in list_name:
    print(item)
```

Syntax Explanation

- `[]` is used to create a list.
- `append()` adds a new element to the end of the list.
- `list_name[index]` updates an existing element.
- `remove()` removes a specified element from the list.
- A `for` loop is used to access each element in the list.

Example

Example: A list can be used to store and manage student names in a classroom.

Code

```
# List of student names
students = ["Avinash", "Rahul", "Priya"]

# Add a new student
students.append("Kiran")

# Update an element
students[1] = "Rohit"

# Remove an element
students.remove("Priya")

print("Students List:")

for student in students:
    print(student)
```

Output

 List Output

Key Takeaway

Lists are flexible and easy to use, making them suitable for storing collections of data that may change over time.

2. Tuples

Diagram

 Tuple Diagram

Explanation

A Tuple is an ordered collection of elements that cannot be modified after creation. Unlike Lists, Tuples are immutable, which means elements cannot be added, removed, or changed.

Tuples can store multiple data types and maintain the order of insertion. They are commonly used to store fixed data that should remain unchanged throughout program execution.

Characteristics of Tuples

- Ordered collection of elements
- Immutable (cannot be modified)
- Allows duplicate values
- Supports indexing and slicing
- Can store multiple data types
- Faster than Lists for fixed data

Syntax

```
tuple_name = (value1, value2, value3)

print(tuple_name[index])
```

Syntax Explanation

- `()` is used to create a tuple.
- Values inside a tuple are separated by commas.
- Each element has an index starting from `0`.
- Elements are accessed using their index.
- Tuples are **immutable**, meaning their values cannot be changed after creation.

Example

Example: Employee details such as Employee ID, Name, Department, and Location can be stored using a Tuple because this information is fixed and rarely changes.

Code

```
employee = (101, "Avinash", "AI/ML", "Bangalore")

print("Employee Details")
print("Employee ID:", employee[0])
print("Name:", employee[1])
print("Department:", employee[2])
print("Location:", employee[3])
```

Code Explanation

Step 1

```
employee = (101, "Avinash", "AI/ML", "Bangalore")
```

Creates a tuple named `employee` containing four values:

- Employee ID
 - Name
 - Department
 - Location
-

Step 2

```
print("Employee Details")
```

Displays a heading before printing the employee information.

Step 3

```
print("Employee ID:", employee[0])
```

Accesses the first element of the tuple using index `0`.

Step 4

```
print("Name:", employee[1])
```

Accesses the second element using index `1`.

Step 5

```
print("Department:", employee[2])
```

Accesses the third element using index `2`.

Step 6

```
print("Location:", employee[3])
```

Accesses the fourth element using index 3.

Output

 Tuple Output

Key Takeaway

A **Tuple** is an **ordered and immutable** collection used to store multiple values. Once a tuple is created, its elements cannot be modified, making it suitable for storing fixed data such as employee details, dates, or coordinates.

3. Sets

Diagram

 Set Diagram

Explanation

A Set is an unordered collection of unique elements. Unlike Lists and Tuples, Sets do not allow duplicate values. They are useful when storing unique data and performing operations such as union, intersection, and difference.

Since Sets are unordered, elements do not have a fixed position and cannot be accessed using indexes.

Characteristics of Sets

- Unordered collection
- Stores unique values only
- Mutable (can add or remove elements)
- Does not support indexing
- Efficient for membership testing

Syntax

```
set_name = {value1, value2, value3}

set_name.add(value)
set_name.remove(value)
```

```
print(set_name)
print(len(set_name))
```

Syntax Explanation

- `{}` is used to create a set.
- A set stores only **unique** values.
- Duplicate values are automatically removed.
- `add()` adds a new element to the set.
- `remove()` removes an element from the set.
- `len()` returns the number of unique elements in the set.

Example

Example: A Set can be used to store unique visitor IDs on a website to avoid duplicate entries.

Code

```
visitor_ids = {101, 102, 103, 101, 102, 104}

print("Unique Visitor IDs:")
print(visitor_ids)

print("\nTotal Unique Visitors:", len(visitor_ids))
```

Code Explanation

Step 1

```
visitor_ids = {101, 102, 103, 101, 102, 104}
```

Creates a set named `visitor_ids`.

Although duplicate values (`101` and `102`) are included, the set automatically removes them.

Step 2

```
print("Unique Visitor IDs:")
```

Displays a heading before printing the set.

Step 3

```
print(visitor_ids)
```

Prints all unique visitor IDs stored in the set.

Step 4

```
print("\nTotal Unique Visitors:", len(visitor_ids))
```

Uses the `len()` function to count the total number of unique visitor IDs in the set.

Output

 Set Output

Key Takeaway

A **Set** is an **unordered** collection that stores **unique** elements. It automatically removes duplicate values, making it ideal for storing unique data such as visitor IDs, email addresses, or product codes.

4. Dictionaries

Diagram

 Dictionary Diagram

Explanation

A Dictionary is a collection of data stored as key-value pairs. Each key is unique and is used to access its corresponding value.

Dictionaries are widely used when data needs to be stored and retrieved using meaningful identifiers instead of numeric indexes.

Characteristics of Dictionaries

- Stores data as key-value pairs
- Keys must be unique
- Mutable (can be updated)
- Fast data retrieval using keys

- Can store multiple data types

Syntax

```
dictionary_name = {  
    key1: value1,  
    key2: value2  
}  
  
print(dictionary_name[key])  
  
dictionary_name[new_key] = new_value  
  
for key, value in dictionary_name.items():  
    print(key, value)
```

Syntax Explanation

- `{}` is used to create a dictionary.
- Each element is stored as a **key-value pair**.
- Keys must be **unique**.
- Values are accessed using their corresponding keys.
- A new key-value pair can be added using `dictionary_name[new_key] = new_value`.
- The `items()` method returns both keys and values for iteration.

Example

Example: A Dictionary can be used to store student information where the Student ID acts as the key and the student name acts as the value.

Code

```
students = {  
    101: "Avinash",  
    102: "Rahul",  
    103: "Priya"  
}  
  
print("Student Records")  
print("Student ID 101:", students[101])  
  
students[104] = "Kiran"  
  
print("\nAll Students:")  
for student_id, name in students.items():  
    print(student_id, "-", name)
```

Code Explanation

Step 1

```
students = {  
    101: "Avinash",  
    102: "Rahul",  
    103: "Priya"  
}
```

Creates a dictionary named `students` that stores student IDs as **keys** and student names as **values**.

Step 2

```
print("Student Records")
```

Displays a heading before printing the student information.

Step 3

```
print("Student ID 101:", students[101])
```

Retrieves the value associated with the key `101`.

Step 4

```
students[104] = "Kiran"
```

Adds a new key-value pair to the dictionary.

- Key: 104
- Value: "Kiran"

Step 5

```
for student_id, name in students.items():  
    print(student_id, "-", name)
```

Uses the `items()` method to iterate through all key-value pairs and print each student's ID and name.

Output

 Dictionary Output

Key Takeaway

A **Dictionary** stores data as **key-value pairs**. It allows fast retrieval, addition, and modification of values using unique keys, making it ideal for storing structured data such as student records, employee information, and product details.

5. Stack

Diagram

 Stack Diagram

Explanation

A Stack is a linear data structure that follows the **Last In, First Out (LIFO)** principle. This means the last element added to the stack is the first element removed.

You can think of a stack like a pile of plates. New plates are added to the top, and plates are also removed from the top.

Common Stack Operations

- **Push** – Add an element to the top of the stack
- **Pop** – Remove the top element from the stack

- **Peek** – View the top element without removing it
- **Size** – Get the number of elements in the stack

Syntax

```
stack = []

# Push
stack.append(item)

# Peek
print(stack[-1])

# Size
print(len(stack))

# Pop
stack.pop()
```

Syntax Explanation

- `[]` is used to create an empty stack.
 - `append()` adds an element to the top of the stack (**Push**).
 - `stack[-1]` accesses the top element without removing it (**Peek**).
 - `len()` returns the number of elements in the stack (**Size**).
 - `pop()` removes and returns the top element (**Pop**).
-

Example

Example: Browser Back Button functionality uses a Stack. The most recently visited page is the first page returned when the Back button is pressed.

Code

```
stack = []

# Push
stack.append("Home Page")
stack.append("Products Page")
stack.append("Checkout Page")

# Peek
print("Current Page:", stack[-1])

# Size
print("Stack Size:", len(stack))
```

```
# Pop
print("Going Back To:", stack.pop())

print("Current Stack:", stack)
```

Code Explanation

Step 1

```
stack = []
```

Creates an empty stack.

Step 2

```
stack.append("Home Page")
stack.append("Products Page")
stack.append("Checkout Page")
```

Adds three pages to the stack using the **Push** operation.

The last page added (**Checkout Page**) is placed on the top of the stack.

Step 3

```
print("Current Page:", stack[-1])
```

Displays the top element of the stack using the **Peek** operation without removing it.

Step 4

```
print("Stack Size:", len(stack))
```

Displays the total number of elements currently stored in the stack.

Step 5

```
print("Going Back To:", stack.pop())
```

Removes and returns the top element of the stack using the **Pop** operation.

Since a stack follows the **Last In, First Out (LIFO)** principle, the last page added is removed first.

Step 6

```
print("Current Stack:", stack)
```

Displays the remaining elements after performing the pop operation.

Output

```
Current Page: Checkout Page
Stack Size: 3
Going Back To: Checkout Page
Current Stack: ['Home Page', 'Products Page']
```

 Stack Output

Key Takeaway

- A **Stack** follows the **Last In, First Out (LIFO)** principle.
 - Elements are added using **Push** (`append()`).
 - The top element is viewed using **Peek** (`stack[-1]`).
 - Elements are removed using **Pop** (`pop()`).
 - `len()` returns the number of elements in the stack.
 - Stacks are commonly used in browser history, undo operations, expression evaluation, and function call management.
-

6. Queue

Diagram

 Queue Diagram

Explanation

A Queue is a linear data structure that follows the **First In, First Out (FIFO)** principle. This means the first element added to the queue is the first element removed.

A queue works like a line of people waiting for a service. The person who joins the line first will be served first.

Common Queue Operations

- **Enqueue** – Add an element to the rear of the queue
- **Dequeue** – Remove an element from the front of the queue
- **Front** – View the first element without removing it
- **Size** – Get the number of elements in the queue

Syntax

```
from collections import deque

queue = deque()

# Enqueue
queue.append(item)

# Front
print(queue[0])

# Size
print(len(queue))

# Dequeue
queue.popleft()
```

Syntax Explanation

- `deque()` creates an empty queue.
- `append()` adds an element to the rear of the queue (**Enqueue**).
- `queue[0]` accesses the first element without removing it (**Front**).
- `len()` returns the number of elements in the queue (**Size**).
- `popleft()` removes and returns the first element from the queue (**Dequeue**).

Example

 Queue Real-Time Example

Example: A ticket booking counter follows the FIFO principle. The customer who joins the queue first receives the ticket first.

Code

```
from collections import deque

queue = deque()

# Enqueue
queue.append("Customer 1")
queue.append("Customer 2")
queue.append("Customer 3")

# Front
print("First Customer:", queue[0])

# Size
print("Queue Size:", len(queue))

# Dequeue
print("Served:", queue.popleft())

print("Current Queue:", list(queue))
```

Code Explanation

Step 1

```
from collections import deque
```

Imports the `deque` class from Python's `collections` module to implement a queue efficiently.

Step 2

```
queue = deque()
```

Creates an empty queue.

Step 3

```
queue.append("Customer 1")
queue.append("Customer 2")
queue.append("Customer 3")
```

Adds three customers to the queue using the **Enqueue** operation.

Customers are added to the rear of the queue.

Step 4

```
print("First Customer:", queue[0])
```

Displays the first customer in the queue using the **Front** operation without removing them.

Step 5

```
print("Queue Size:", len(queue))
```

Displays the total number of customers currently waiting in the queue.

Step 6

```
print("Served:", queue.popleft())
```

Removes and returns the first customer from the queue using the **Dequeue** operation.

Since a queue follows the **First In, First Out (FIFO)** principle, the first customer who joined the queue is served first.

Step 7

```
print("Current Queue:", list(queue))
```

Displays the remaining customers after serving the first customer.

Output

 Queue Output

Key Takeaway

- A **Queue** follows the **First In, First Out (FIFO)** principle.
- Elements are added using **Enqueue** (`append()`).
- The first element is viewed using **Front** (`queue[0]`).

- Elements are removed using **Dequeue** (`popleft()`).
 - `len()` returns the number of elements in the queue.
 - Queues are widely used in ticket booking systems, customer service, task scheduling, printer queues, and CPU scheduling.
-

7. Linked List

Diagram

 Linked List Diagram

Explanation

A Linked List is a linear data structure made up of nodes. Each node contains two parts:

- Data – stores the actual value
- Next Pointer – stores the address of the next node

Unlike arrays, linked list elements are not stored in contiguous memory locations. Each node is connected to the next node through links.

Characteristics of Linked Lists

- Dynamic size
- Efficient insertion and deletion
- Nodes are connected using pointers
- Elements are not stored in contiguous memory
- Sequential access only

Types of Linked Lists

1. Singly Linked List

A singly linked list is the simplest kind of linked lists. It takes up less space in memory because each node has only one address to the next node, like in the image below.

 Single Linked List Diagram

2. Doubly Linked List

A doubly linked list has nodes with addresses to both the previous and the next node, like in the image below, and therefore takes up more memory. But doubly linked lists are good if you want to be able to move both up and down in the list.

 doubly Linked List Diagram

3. Circular Linked List

A circular linked list is like a singly or doubly linked list with the first node, the "head", and the last node, the "tail", connected.

In singly or doubly linked lists, we can find the start and end of a list by just checking if the links are null. But for circular linked lists, more complex code is needed to explicitly check for start and end nodes in certain applications.

Circular linked lists are good for lists you need to cycle through continuously.

The image below is an example of a singly circular linked list:

Circular Linked List Diagram

The image below is an example of a doubly circular linked list:

Circular Linked List Diagram

Syntax

```
class Node:

    def __init__(self, data):
        self.data = data
        self.next = None

head = Node(value1)
head.next = Node(value2)

current = head

while current:
    print(current.data)
    current = current.next
```

Syntax Explanation

- `class Node:` defines a node of the linked list.
- `self.data` stores the value of the node.
- `self.next` stores the reference to the next node.
- `head` represents the first node of the linked list.
- Nodes are connected using the `next` reference.
- A `while` loop is used to traverse the linked list until the last node is reached.

Example

Example: A music playlist can be represented using a Linked List, where each song points to the next song in the playlist.

Linked List Operations

1. Traversal

Traversal means visiting each node in the linked list one by one.

Example:

```
current = head

while current:
    print(current.data)
    current = current.next
```

Output

```
Song A
Song B
Song C
```

2. Insertion

Insertion means adding a new node into the linked list.

Example:

```
new_node = Node("Song D")

new_node.next = song2.next
song2.next = new_node
```

Result

```
Song A → Song B → Song D → Song C
```

3. Deletion

Deletion means removing a node from the linked list.

Example:

```
song2.next = song3.next
```

Result

Song A → Song B → NULL

4. Searching

Searching means finding a specific node in the linked list.

Example:

```
current = head

while current:
    if current.data == "Song B":
        print("Song Found")
        break
    current = current.next
```

Output

Song Found

5. Sorting

Sorting arranges nodes in ascending or descending order.

Example:

Before Sorting:

30 → 10 → 20

After Sorting:

10 → 20 → 30

Code

```
class Node:
    def __init__(self, song):
        self.song = song
        self.next = None

song1 = Node("Song A")
song2 = Node("Song B")
song3 = Node("Song C")

song1.next = song2
song2.next = song3

current = song1

print("Playlist:")

while current:
    print(current.song)
    current = current.next
```

Code Explanation

Step 1

```
class Node:
```

Defines a `Node` class. Each node represents one song in the playlist.

Step 2

```
def __init__(self, song):
    self.song = song
    self.next = None
```

Initializes each node with:

- `song` → Stores the song name.
- `next` → Stores a reference to the next node. Initially, it is `None`.

Step 3

```
song1 = Node("Song A")
song2 = Node("Song B")
song3 = Node("Song C")
```

Creates three nodes representing three songs in the playlist.

Step 4

```
song1.next = song2
song2.next = song3
```

Links the nodes together.

The linked list becomes:

```
Song A → Song B → Song C → NULL
```

Step 5

```
current = song1
```

Assigns the first node (`song1`) to the variable `current`.

This variable is used to traverse the linked list.

Step 6

```
print("Playlist:")
```

Displays a heading before printing the playlist.

Step 7

```
while current:
    print(current.song)
    current = current.next
```

Traverses the linked list.

For each iteration:

- Prints the current song.
- Moves to the next node using `current.next`.

The loop stops automatically when `current` becomes `None`, indicating the end of the linked list.

Output

```
Playlist:
Song A
Song B
Song C
```

 Linked List Output

Key Takeaway

- A **Linked List** stores data in nodes connected using links.
 - Each node contains **data** and a reference to the **next node**.
 - Linked Lists support efficient insertion and deletion without shifting elements.
 - Traversal is performed by following the `next` reference from one node to another.
 - Linked Lists are commonly used in playlists, browser history, memory management, and dynamic data structures.
-

8. Deque (Double-Ended Queue)

Diagram

 Deque Diagram

Explanation

A Deque (Double-Ended Queue) is a data structure that allows insertion and deletion of elements from both the front and rear ends.

Unlike a Queue, which allows insertion at one end and deletion at the other, a Deque supports operations at both ends efficiently.

Characteristics of Deque

- Insertion from both ends
- Deletion from both ends

- Can work as both Queue (FIFO) and Stack (LIFO)
- Efficient for real-time data processing
- Implemented using `collections.deque`

Common Deque Operations

- `append()` → Add element to the right end
- `appendleft()` → Add element to the left end
- `pop()` → Remove element from the right end
- `popleft()` → Remove element from the left end

Syntax

```
from collections import deque

deque_name = deque()

# Add elements
deque_name.append(item)
deque_name.appendleft(item)

# Access elements
print(deque_name[0])
print(deque_name[-1])

# Remove elements
deque_name.pop()
deque_name.popleft()
```

Syntax Explanation

- `deque()` creates an empty deque.
- `append()` adds an element to the **right end** of the deque.
- `appendleft()` adds an element to the **left end** of the deque.
- `deque_name[0]` accesses the first element.
- `deque_name[-1]` accesses the last element.
- `pop()` removes and returns the element from the **right end**.
- `popleft()` removes and returns the element from the **left end**.

Example

Example: Browser history uses a Deque because users can move backward and forward between pages.

`append()` – Add element to the right end

```
from collections import deque

dq = deque(["Google", "YouTube"])

dq.append("GitHub")

print(dq)
```

Output

```
deque(['Google', 'YouTube', 'GitHub'])
```

appendleft() – Add element to the left end

```
from collections import deque

dq = deque(["Google", "YouTube"])

dq.appendleft("Home")

print(dq)
```

Output

```
deque(['Home', 'Google', 'YouTube'])
```

pop() – Remove element from the right end

```
from collections import deque

dq = deque(["Google", "YouTube", "GitHub"])

dq.pop()

print(dq)
```

Output

```
deque(['Google', 'YouTube'])
```

popleft() – Remove element from the left end

```
from collections import deque

dq = deque(["Home", "Google", "YouTube"])

dq.popleft()

print(dq)
```

Output

```
deque(['Google', 'YouTube'])
```

Code

```
from collections import deque

history = deque()

history.append("Google")
history.append("YouTube")

history.appendleft("Home")

print("Browser History:")
print(history)

history.pop()

history.popleft()

print("\nUpdated History:")
print(history)
```

Code Explanation

Step 1

```
from collections import deque
```

Imports the `deque` class from Python's `collections` module.

Step 2

```
history = deque()
```

Creates an empty deque named `history`.

Step 3

```
history.append("Google")  
history.append("YouTube")
```

Adds two web pages to the **right end** of the deque using the `append()` method.

Current deque:

```
Google ← YouTube
```

Step 4

```
history.appendleft("Home")
```

Adds `"Home"` to the **left end** of the deque using the `appendleft()` method.

Current deque:

```
Home ← Google ← YouTube
```

Step 5

```
print("Browser History:")
```

```
print(history)
```

Displays the current browser history.

Step 6

```
history.pop()
```

Removes the last page (YouTube) from the **right end** of the deque.

Step 7

```
history.popleft()
```

Removes the first page (Home) from the **left end** of the deque.

The remaining deque is:

```
Google
```

Step 8

```
print("\nUpdated History:")  
print(history)
```

Displays the updated browser history after both removal operations.

Output

```
Browser History:  
deque(['Home', 'Google', 'YouTube'])  
  
Updated History:  
deque(['Google'])
```

 Deque Output

Key Takeaway

- A **Deque (Double-Ended Queue)** allows insertion and deletion from **both the left and right ends**.
 - `append()` adds an element to the right end.
 - `appendleft()` adds an element to the left end.
 - `pop()` removes an element from the right end.
 - `popleft()` removes an element from the left end.
 - Deques are ideal for browser history, task scheduling, sliding window algorithms, and implementing both stacks and queues efficiently.
-

9. Priority Queue

Explanation

A Priority Queue is a special type of queue where elements are processed based on their priority rather than their order of arrival.

Elements with higher priority are served before elements with lower priority.

Characteristics of Priority Queue

- Elements are processed according to priority
- Higher priority elements are served first
- If priorities are equal, elements are served in insertion order
- Useful for task scheduling and resource management

Key differences between priority queue and queue

 priority Diagram

Applications of priority queue

Let's understand the applications of a priority queue because they demonstrate how this data structure can be utilized in real-world scenarios to manage tasks efficiently. Here are some key applications:

- Task Scheduling (Operating Systems) manages tasks by priority, executing high-priority tasks first in real-time systems.
- Dijkstra's Shortest Path Algorithm uses a priority queue to find the shortest path by selecting the nearest node.
- Huffman Encoding (Data Compression) combines least frequent symbols using a priority queue to reduce data size.
- Merging Multiple Sorted Lists merges sorted lists by selecting the smallest element from each list.
- A Search Algorithm (Pathfinding) prioritizes nodes based on cost to find the shortest path in navigation or games.

Types of Priority Queue

Max Priority Queue

The element with the highest priority is processed first.

Min Priority Queue

The element with the lowest priority is processed first.

Syntax

```
import heapq

priority_queue = []

# Insert elements
heapq.heappush(priority_queue, (priority, value))

# Remove highest-priority element
heapq.heappop(priority_queue)
```

Syntax Explanation

- `import heapq` imports Python's built-in module for implementing a Priority Queue.
- `priority_queue = []` creates an empty priority queue.
- `heappush()` inserts an element into the priority queue.
- Each element is stored as a tuple `(priority, value)`.
- Smaller priority values are served first.
- `heappop()` removes and returns the element with the highest priority (lowest priority number).

Example

Example: In a hospital emergency room, patients with critical conditions are treated before patients with minor injuries.

Code

```
import heapq

patients = []

heapq.heappush(patients, (1, "Critical Patient"))
heapq.heappush(patients, (2, "Serious Patient"))
heapq.heappush(patients, (3, "Normal Patient"))
```

```
print("Patient Being Treated:")

while patients:
    print(heapq.heappop(patients)[1])
```

Code Explanation

Step 1

```
import heapq
```

Imports Python's built-in `heapq` module, which provides functions for implementing a Priority Queue.

Step 2

```
patients = []
```

Creates an empty priority queue named `patients`.

Step 3

```
heapq.heappush(patients, (1, "Critical Patient"))
heapq.heappush(patients, (2, "Serious Patient"))
heapq.heappush(patients, (3, "Normal Patient"))
```

Adds patients to the priority queue.

Each patient is stored as a tuple:

- First value → Priority
- Second value → Patient Name

Lower priority numbers indicate higher priority.

Current queue:

```
(1, Critical Patient)
(2, Serious Patient)
(3, Normal Patient)
```

Step 4

```
print("Patient Being Treated:")
```

Displays a heading before treating patients.

Step 5

```
while patients:
```

Continues processing until the priority queue becomes empty.

Step 6

```
print(heapq.heappop(patients)[1])
```

Removes and returns the patient with the highest priority.

- `heappop()` removes the tuple with the smallest priority number.
- `[1]` retrieves only the patient's name from the tuple.

Patients are treated in this order:

1. Critical Patient
 2. Serious Patient
 3. Normal Patient
-

Output

```
Patient Being Treated:  
Critical Patient  
Serious Patient  
Normal Patient
```

 Priority Queue Output

Key Takeaway

- A **Priority Queue** processes elements based on **priority** rather than insertion order.
- `heappush()` inserts elements into the priority queue.

- `heappop()` removes the element with the **highest priority** (lowest priority value).
- Elements are commonly stored as `(priority, value)` tuples.
- Priority Queues are widely used in hospital emergency systems, CPU scheduling, task scheduling, pathfinding algorithms, and event-driven applications.

10. Hash Table

Explanation

A **Hash Table** is a data structure that stores data as **key-value pairs** and allows very fast searching, insertion, deletion, and updating of data.

It uses a **Hash Function** to convert a key into an index where the corresponding value is stored. This enables data to be accessed quickly without searching through every element.

In Python, the built-in **Dictionary** (`dict`) is implemented using a **Hash Table**, making dictionary operations very efficient.

Why Use a Hash Table?

- Stores data as key-value pairs
- Provides fast searching
- Allows quick insertion and deletion
- Efficient for large datasets
- Widely used in real-world applications

Syntax

```
hash_table = {  
    key1: value1,  
    key2: value2  
}  
  
print(hash_table[key])  
  
hash_table[new_key] = new_value  
  
for key, value in hash_table.items():  
    print(key, value)
```

Syntax Explanation

- `{}` is used to create a Hash Table (Dictionary) in Python.
- Data is stored as **key-value pairs**.
- Keys must be **unique**.

- `hash_table[key]` retrieves the value associated with a key.
 - `hash_table[new_key] = new_value` adds a new key-value pair.
 - `items()` returns all key-value pairs for iteration.
-

Understanding How a Hash Table Works

A Hash Table stores data using a **Hash Function**.

Instead of searching every element one by one, the Hash Function calculates the location where the data should be stored.

The overall process is shown below.

```
Key
↓
Hash Function
↓
Generate Index
↓
Store Value
↓
Retrieve Value Quickly
```

Let's understand each step.

Step 1: Create Buckets

A Hash Table contains storage locations called **Buckets**.

```
hash_table = [None] * 10
```

Here, ten empty buckets are created to store data.

Step 2: Apply the Hash Function

The Hash Function converts a key into an index.

```
def hash_function(key):
    return len(key) % 10
```

For example,

```
Key = "Bob"

Length = 3

Index = 3 % 10 = 3
```

The value will be stored in bucket **3**.

Step 3: Insert Data

Store the value at the calculated index.

```
index = hash_function("Bob")
hash_table[index] = "Bob"
```

Now the value "Bob" is stored in bucket 3.

Step 4: Search Data

To retrieve the value, Python applies the same Hash Function.

```
index = hash_function("Bob")
print(hash_table[index])
```

Since the index is already known, searching is very fast.

Step 5: Handle Collisions

Sometimes two different keys generate the same index.

This situation is called a **Collision**.

Example:

```
Lisa → Bucket 3
Stuart → Bucket 3
```

Both values want to occupy the same bucket.

One common solution is **Chaining**, where multiple values are stored in the same bucket.

```
hash_table = [  
    [],  
    [],  
    [],  
    ["Lisa", "Stuart"],  
    [],  
    []  
]
```

Example

Example: A phone contact directory stores names as keys and phone numbers as values.

Code

```
contacts = {  
    "Avinash": "9876543210",  
    "Rahul": "9876543211",  
    "Priya": "9876543212"  
}  
  
print("Avinash:", contacts["Avinash"])  
  
contacts["Kiran"] = "9876543213"  
  
print("\nAll Contacts:")  
  
for name, number in contacts.items():  
    print(name, "-", number)
```

Code Explanation

Step 1

```
contacts = {  
    "Avinash": "9876543210",  
    "Rahul": "9876543211",  
    "Priya": "9876543212"  
}
```

Creates a Hash Table named `contacts`.

Each entry consists of:

- **Key** → Contact Name
 - **Value** → Phone Number
-

Step 2

```
print("Avinash:", contacts["Avinash"])
```

Retrieves the phone number associated with the key "Avinash".

Step 3

```
contacts["Kiran"] = "9876543213"
```

Adds a new contact to the Hash Table.

- Key → "Kiran"
 - Value → "9876543213"
-

Step 4

```
print("\nAll Contacts:")
```

Displays a heading before printing all contacts.

Step 5

```
for name, number in contacts.items():  
    print(name, "-", number)
```

Iterates through all key-value pairs using the `items()` method and displays every contact stored in the Hash Table.

Output

```
Avinash: 9876543210
```

```
All Contacts:  
Avinash - 9876543210
```

Rahul - 9876543211

Priya - 9876543212

Kiran - 9876543213

Applications of Hash Tables

- Phone Contact Directories
- Database Indexing
- Password Storage
- Caching Systems
- Search Engines
- DNS Lookup
- Symbol Tables in Compilers

Key Takeaway

- A **Hash Table** stores data as **key-value pairs**.
- It uses a **Hash Function** to determine where data should be stored.
- Searching, insertion, deletion, and updating are very fast.
- In Python, **Dictionaries (dict)** are implemented using Hash Tables.
- Hash Tables are one of the most widely used data structures in modern software development.

Data Structures Summary

Data Structures help organize and manage data efficiently. Choosing the right data structure improves performance, reduces complexity, and makes applications more scalable.

The key concepts covered in this document are:

- Lists
- Tuples
- Sets
- Dictionaries
- Stacks
- Queues
- Linked Lists
- Deques
- Priority Queues
- Hash Tables

Each data structure has its own advantages and use cases. Understanding these structures helps developers select the most suitable approach for solving real-world problems.

Decorators

- [Overview](#)
 - [Why Use Decorators?](#)
 - [How Decorators Work](#)
 - [Functions as First-Class Objects](#)
 - [Higher-Order Functions](#)
 - [Types of Decorators](#)
 - [Basic Decorator](#)
 - [Decorator with Arguments](#)
 - [functools.wraps Example](#)
 - [Timer Decorator](#)
 - [Logging Decorator](#)
 - [Summary](#)
-

Decorators Overview

 decorators overview Decorators are a powerful feature in Python that allow you to **add new functionality to an existing function without modifying its original code**.

A decorator is simply a function that takes another function as input, adds some additional behavior, and returns the modified function.

Think of a decorator as a **wrapper** around a function. The original function remains unchanged, but extra functionality is executed before, after, or around it.

Understanding the Image

The image above shows a **red apple** being transformed into a **green apple**.

Similarly, a decorator transforms the behavior of a function without changing the function itself.

-  **Red Apple** → Original Function
- → **Decorator** → Wraps the original function and adds extra functionality
-  **Green Apple** → Enhanced Function with additional behavior

The original function still exists, but after applying a decorator, it gains new capabilities.

For example:

- Before executing a function, a decorator can display a message.
- After executing a function, it can log the result.
- It can measure execution time.
- It can validate user input.
- It can check whether a user is authenticated.

In all these cases, **the original function remains unchanged**, while the decorator adds the required functionality.

Why Use Decorators?

Decorators are widely used because they help developers avoid writing the same code repeatedly and keep programs clean and organized.

Code Reusability

A single decorator can be applied to multiple functions, eliminating duplicate code.

Cleaner Code

The main function contains only its core logic, while additional tasks are handled separately by decorators.

Logging and Monitoring

Decorators can automatically record function calls, execution details, and application events.

Authentication and Authorization

Decorators can verify whether a user has permission before allowing access to a function.

Performance Measurement

Decorators can calculate and display how long a function takes to execute.

Input Validation

Decorators can validate function arguments before the function is executed.

By separating these common tasks from the main business logic, decorators make Python programs more modular, reusable, and easier to maintain.

Functions as First-Class Objects

In Python, functions are first-class objects. This means they can:

- Be assigned to variables
- Be passed as arguments
- Be returned from other functions
- Be stored in data structures

Higher-Order Functions

A Higher-Order Function is a function that:

- Accepts another function as an argument
- Returns a function as a result

Decorators are built using higher-order functions.

How Decorators Work



A decorator works by **wrapping an existing function** and adding extra functionality without modifying the original function's code.

Instead of changing the original function directly, Python passes the function to a decorator. The decorator creates a **wrapper function**, adds additional behavior, and then returns the wrapper as the new function.

The original function remains unchanged, but whenever it is called, the wrapper function executes first.

Workflow

```
Original Function
    ↓
Passed to Decorator
    ↓
Decorator Creates Wrapper
    ↓
Adds Extra Functionality
(Before / After Execution)
    ↓
Returns Wrapped Function
    ↓
Decorated Function Executes
```

Step-by-Step Explanation

Step 1: Original Function

First, we create a normal Python function.

```
def greet():
    print("Hello")
```

At this stage, calling `greet()` simply prints:

```
Hello
```

Step 2: Function is Passed to the Decorator

When we apply a decorator using the `@decorator` syntax, Python automatically passes the original function to the decorator.

```
greet()
  ↓
decorator(greet)
```

Here, `greet()` becomes the input to the `decorator()` function.

Step 3: Decorator Creates a Wrapper

Inside the decorator, a new function called `wrapper()` is created.

The wrapper contains the additional functionality that should run before or after the original function.

```
Decorator
  ↓
Creates wrapper()
```

Note: Are `decorator` and `wrapper` Special Names?

No. The names `decorator` and `wrapper` are **not Python keywords**. They are simply function names chosen by the programmer.

You can replace them with any meaningful names.

For example:

```
def logger(func):
    def execute():
        ...
    return execute
```

Step 4: Wrapper Calls the Original Function

The wrapper first performs the additional task, then calls the original function.

```
wrapper()
  ↓
Before Message
  ↓
Original Function
  ↓
After Message
```

Step 5: Decorated Function Executes

Finally, the decorator returns the wrapper function.

Now, whenever we call:

```
greet()
```

Python actually executes:

```
wrapper()  
  ↓  
Before  
  ↓  
Hello  
  ↓  
After
```

The original `greet()` function is never modified, but its behavior is enhanced through the decorator.

Examples

Concept	File
Basic Decorator	01_basic_decorator.py
Decorator with Arguments	02_decorator_with_args.py
functools.wraps Example	03_wraps_example.py
Timer Decorator	04_timer_decorator.py
Logging Decorator	05_logging_decorator.py

1. Basic Decorator

Explanation

A Basic Decorator is used to add extra functionality to a function without modifying its original code.

Instead of changing the original function, a decorator wraps it and executes additional code before or after the function runs.

Decorators help keep code clean, reusable, and easier to maintain.

Syntax

The general syntax of a basic decorator is:

```
def decorator(func):  
  
    def wrapper():  
        # Code to execute before the original function  
  
        func()  
  
        # Code to execute after the original function  
  
    return wrapper  
  
@decorator  
def function_name():  
    # Original function
```

Syntax Explanation

- `decorator(func)` receives the original function as an argument.
- `wrapper()` contains the additional functionality.
- `func()` calls the original function.
- `return wrapper` returns the wrapped function.
- `@decorator` applies the decorator to the function.

Code Flow

The following flow shows how a decorator works internally.

```
Define Decorator  
  ↓  
Define Original Function  
  ↓  
Apply @decorator  
  ↓  
Decorator Receives Function  
  ↓  
Creates wrapper()  
  ↓  
Returns wrapper()  
  ↓  
Call Function  
  ↓  
wrapper() Executes  
  ↓  
Before Code  
  ↓  
Original Function
```

↓
After Code

How Python Executes

When Python sees the following code:

```
@decorator
def greet():
    print("Hello, World!")
```

Python automatically converts it into:

```
def greet():
    print("Hello, World!")

greet = decorator(greet)
```

Now, when the following statement is executed:

```
greet()
```

Python actually executes:

```
wrapper()
  ↓
Before calling the function
  ↓
Original greet() function
  ↓
Hello, World!
  ↓
After calling the function
```

The original function is not modified. Instead, it is replaced with the wrapped version returned by the decorator.

Example

Example: A decorator can display a message before and after executing a function without changing the function's original implementation.

Code

```
def decorator(func):  
  
    def wrapper():  
        print("Before calling the function")  
  
        func()  
  
        print("After calling the function")  
  
    return wrapper  
  
@decorator  
def greet():  
    print("Hello, World!")  
  
greet()
```

Code Explanation

Step 1

```
def decorator(func):
```

Defines a decorator function that accepts another function as its argument.

Here, `func` refers to the original function (`greet`).

Step 2

```
def wrapper():
```

Creates an inner function called `wrapper()`.

This function is responsible for adding extra functionality.

Step 3

```
print("Before calling the function")
```

Executes before the original function.

Step 4

```
func()
```

Calls the original `greet()` function.

Step 5

```
print("After calling the function")
```

Executes after the original function finishes.

Step 6

```
return wrapper
```

Returns the wrapper function.

This replaces the original function with the wrapped version.

Step 7

```
@decorator
```

Applies the decorator to the `greet()` function.

Internally, Python performs:

```
greet = decorator(greet)
```

Step 8

```
greet()
```

Calling `greet()` actually executes the `wrapper()` function, which then calls the original function.

Output

```
Before calling the function
Hello, World!
After calling the function
```

 Basic Decorator Output

Key Takeaway

A Basic Decorator wraps an existing function and adds extra functionality before or after the original function executes without modifying its original implementation. Decorators improve code reusability, readability, and maintainability by separating common tasks from the main business logic.

2. Decorator with Arguments

Explanation

Sometimes, the function being decorated accepts one or more arguments. In such cases, the decorator must also be able to receive and pass those arguments to the original function.

Python provides `*args` and `**kwargs` to make decorators flexible enough to work with functions that accept any number of positional and keyword arguments.

Without `*args` and `**kwargs`, the decorator would only work with functions that have no parameters.

Why Use Decorators with Arguments?

A basic decorator works well for functions that do not require parameters.

However, many real-world functions accept arguments.

For example:

```
add(5, 3)
login(username, password)
send_email(receiver, subject)
```

To decorate these functions without knowing how many arguments they receive, we use:

- `*args` → Accepts any number of positional arguments.
- `**kwargs` → Accepts any number of keyword arguments.

This makes the decorator reusable for different types of functions.

Syntax

```
def decorator(func):  
  
    def wrapper(*args, **kwargs):  
  
        # Code before function execution  
  
        result = func(*args, **kwargs)  
  
        # Code after function execution  
  
        return result  
  
    return wrapper  
  
@decorator  
def function_name(arguments):  
    ...
```

Syntax Explanation

- `decorator(func)` receives the original function.
 - `wrapper(*args, **kwargs)` accepts all arguments passed to the function.
 - `func(*args, **kwargs)` forwards those arguments to the original function.
 - `return result` returns the value produced by the original function.
 - `@decorator` applies the decorator to the function.
-

Code Flow

The following flow shows how a decorator works with function arguments.

```
Define Decorator  
  ↓  
Define Function with Arguments  
  ↓  
Apply @decorator  
  ↓  
Decorator Receives Function
```

↓
Creates wrapper(*args, **kwargs)
↓
Call Function with Arguments
↓
wrapper Receives Arguments
↓
Execute Code Before Function
↓
Original Function Executes
↓
Execute Code After Function
↓
Return Result

How Python Executes

When Python sees:

```
@decorator  
def add(a, b):  
    return a + b
```

Python automatically converts it into:

```
def add(a, b):  
    return a + b  
  
add = decorator(add)
```

Now, when the following statement is executed:

```
add(5, 3)
```

Python actually executes:

```
wrapper(5, 3)  
↓  
Before execution  
↓  
func(5, 3)  
↓  
Returns 8  
↓
```

After execution



Returns 8

The original `add()` function is not modified. Instead, it is replaced with the wrapped version returned by the decorator.

Example

Example: A decorator can display messages before and after executing a function that accepts arguments, such as an addition function.

Code

```
def decorator(func):  
    def wrapper(*args, **kwargs):  
        print("Before execution")  
        result = func(*args, **kwargs)  
        print("After execution")  
        return result  
    return wrapper  
  
@decorator  
def add(a, b):  
    return a + b  
  
print("Result:", add(5, 3))
```

Code Explanation

Step 1

```
def decorator(func):
```

Defines a decorator function that accepts another function as its argument.

Here, `func` refers to the original `add()` function.

Step 2

```
def wrapper(*args, **kwargs):
```

Creates a wrapper function that can accept any number of positional and keyword arguments.

- `*args` stores positional arguments.
 - `**kwargs` stores keyword arguments.
-

Step 3

```
print("Before execution")
```

Executes before calling the original function.

Step 4

```
result = func(*args, **kwargs)
```

Calls the original function and forwards all received arguments.

In this example:

```
add(5, 3)
```

becomes:

```
func(5, 3)
```

The original function performs:

```
5 + 3 = 8
```

The returned value is stored in `result`.

Step 5

```
print("After execution")
```

Executes after the original function finishes.

Step 6

```
return result
```

Returns the value produced by the original function.

Without this statement, the decorated function would return `None`.

Step 7

```
@decorator
```

Applies the decorator to the `add()` function.

Internally, Python performs:

```
add = decorator(add)
```

Step 8

```
add(5, 3)
```

Calling `add(5, 3)` actually executes the `wrapper()` function.

The wrapper:

1. Prints the message before execution.
 2. Calls the original `add()` function.
 3. Receives the result (`8`).
 4. Prints the message after execution.
 5. Returns the result.
-

Output

```
Before execution
After execution
Result: 8
```

 Decorator with Arguments Output

Key Takeaway

Decorators with `*args` and `**kwargs` are more flexible than basic decorators because they can work with functions that accept any number of positional and keyword arguments. This makes them suitable for real-world applications where functions often require different types of input parameters.

4. Timer Decorator

Explanation

A Timer Decorator is used to measure how long a function takes to execute.

Instead of modifying the original function, the decorator records the start time before the function runs and the end time after it finishes. The difference between these two times gives the total execution time.

This technique is commonly used to analyze and improve application performance.

 Timer Decorator

Why Use a Timer Decorator?

A Timer Decorator helps developers understand how efficiently a function executes.

It is commonly used for:

- Performance monitoring
 - Application optimization
 - Benchmarking code
 - Measuring API response times
 - Comparing different algorithms
-

Syntax

```
import time

def timer(func):
```

```
def wrapper():  
    start_time = time.time()  
  
    func()  
  
    end_time = time.time()  
  
    print(end_time - start_time)  
  
    return wrapper  
  
@timer  
def function_name():  
    ...
```

Syntax Explanation

- `time.time()` records the current system time.
- `start_time` stores the time before the function executes.
- `func()` calls the original function.
- `end_time` stores the time after execution.
- `end_time - start_time` calculates the execution time.
- `@timer` applies the timer decorator.

Code Flow

```
Define Timer Decorator  
    ↓  
Define Original Function  
    ↓  
Apply @timer  
    ↓  
Call Function  
    ↓  
Record Start Time  
    ↓  
Execute Original Function  
    ↓  
Record End Time  
    ↓  
Calculate Execution Time  
    ↓  
Display Execution Time
```

How Python Executes

When Python sees:

```
@timer
def process_data():
    ...
```

Python internally converts it into:

```
def process_data():
    ...

process_data = timer(process_data)
```

Now, when:

```
process_data()
```

is executed, Python actually runs:

```
wrapper()
  ↓
Record Start Time
  ↓
Original Function Executes
  ↓
Record End Time
  ↓
Calculate Time Difference
  ↓
Display Execution Time
```

The original function is never modified. The timer decorator simply measures how long it takes to execute.

Example

Example: Measure the execution time of a function that processes data.

Code

```
import time

def timer(func):

    def wrapper():

        start_time = time.time()

        func()

        end_time = time.time()

        print(
            "Execution Time:",
            round(end_time - start_time, 2),
            "seconds"
        )

    return wrapper

@timer
def process_data():

    time.sleep(2)

    print("Processing completed")

process_data()
```

Code Explanation

Step 1

```
import time
```

Imports Python's built-in `time` module to measure execution time.

Step 2

```
def timer(func):
```

Defines the timer decorator.

The parameter `func` represents the original function.

Step 3

```
def wrapper():
```

Creates a wrapper function that contains the timing logic.

Step 4

```
start_time = time.time()
```

Records the current time before executing the original function.

Step 5

```
func()
```

Executes the original function.

In this example, it calls:

```
process_data()
```

Step 6

```
end_time = time.time()
```

Records the current time immediately after the function finishes.

Step 7

```
round(end_time - start_time, 2)
```

Calculates the total execution time and rounds it to two decimal places.

Step 8

```
@timer
```

Applies the timer decorator to the `process_data()` function.

Internally, Python performs:

```
process_data = timer(process_data)
```

Step 9

```
process_data()
```

Calling `process_data()` actually executes the wrapper function, which:

1. Records the start time.
2. Executes the original function.
3. Records the end time.
4. Calculates the execution time.
5. Displays the result.

Output

```
Processing completed  
Execution Time: 2.0 seconds
```



Timer Decorator Output

Key Takeaway

A Timer Decorator measures how long a function takes to execute without modifying the original function. It is widely used to monitor performance, compare algorithms, identify slow functions, and optimize applications.

5. Logging Decorator

Explanation

A Logging Decorator is used to record information whenever a function is executed.

Instead of modifying the original function, the decorator logs important details such as when the function starts and when it finishes. This helps developers monitor application behavior, debug issues, and keep track of function execution.

Logging decorators are widely used in production applications to improve monitoring and troubleshooting.

Why Use a Logging Decorator?

A Logging Decorator automatically records function execution without changing the original function.

It is commonly used for:

- Application monitoring
 - Debugging issues
 - Tracking user actions
 - API request logging
 - System auditing
-

Syntax

```
def logger(func):  
    def wrapper():  
        print("Function Started")  
        func()  
        print("Function Completed")  
    return wrapper  
  
@logger  
def function_name():  
    ...
```

Syntax Explanation

- `logger(func)` receives the original function.
- `wrapper()` contains the logging logic.
- The first `print()` logs when the function starts.
- `func()` executes the original function.
- The second `print()` logs when the function completes.

- `@logger` applies the logging decorator.

Code Flow

```
Define Logger Decorator
  ↓
Define Original Function
  ↓
Apply @logger
  ↓
Call Function
  ↓
Log Function Started
  ↓
Execute Original Function
  ↓
Log Function Completed
```

How Python Executes

When Python sees:

```
@logger
def process_order():
    ...
```

Python automatically converts it into:

```
def process_order():
    ...

process_order = logger(process_order)
```

Now, when:

```
process_order()
```

is executed, Python actually runs:

```
wrapper()
  ↓
Log Function Started
```

↓
Original Function Executes
↓
Log Function Completed

The original function remains unchanged while the decorator records its execution.

Example

Example: Log a message whenever a function starts and completes execution.

Code

```
def logger(func):  
    def wrapper():  
        print(f"[LOG] Function '{func.__name__}' started")  
        func()  
        print(f"[LOG] Function '{func.__name__}' completed")  
    return wrapper  
  
@logger  
def process_order():  
    print("Processing customer order...")  
  
process_order()
```

Code Explanation

Step 1

```
def logger(func):
```

Defines the logging decorator.

The parameter `func` refers to the original function.

Step 2

```
def wrapper():
```

Creates the wrapper function that adds logging before and after the original function executes.

Step 3

```
print(f"[LOG] Function '{func.__name__}' started")
```

Displays a log message before executing the original function.

`func.__name__` returns the name of the function being decorated.

For this example, it returns:

```
process_order
```

Step 4

```
func()
```

Calls the original function.

Here it executes:

```
process_order()
```

Step 5

```
print(f"[LOG] Function '{func.__name__}' completed")
```

Displays a log message after the original function finishes.

Step 6

```
return wrapper
```

Returns the wrapper function.

The original function is replaced with the wrapped version.

Step 7

```
@logger
```

Applies the logging decorator.

Internally, Python performs:

```
process_order = logger(process_order)
```

Step 8

```
process_order()
```

Calling `process_order()` actually executes the wrapper function, which:

1. Logs that the function has started.
 2. Executes the original function.
 3. Logs that the function has completed.
-

Output

```
[LOG] Function 'process_order' started  
Processing customer order...  
[LOG] Function 'process_order' completed
```

 Logging Decorator Output

Key Takeaway

A Logging Decorator records information before and after a function executes without modifying the original function. It is widely used for debugging, monitoring, auditing, and tracking application behavior in real-world

Decorators Summary

Decorators are a powerful Python feature that allows developers to extend the behavior of functions without modifying their original implementation.

In this module, we learned that decorators work by wrapping an existing function inside another function and executing additional functionality before or after the original function.

The concepts covered in this module are:

- Basic Decorator
- Decorator with Arguments
- `functools.wraps` Example
- Timer Decorator
- Logging Decorator

Through these examples, we learned how decorators preserve function metadata, work with functions that accept arguments, measure execution time, and record function execution for debugging and monitoring.

Decorators promote cleaner, reusable, and maintainable code by separating common functionality from business logic. They are widely used in Python frameworks and real-world applications for logging, authentication, authorization, performance monitoring, input validation, caching, and many other cross-cutting concerns.

Generators

- [Generators Overview](#)
 - [How Generators Work](#)
 - [Why Use Generators?](#)
 - [Generator vs List](#)
 - [When to Use Generators?](#)
 - [Real-World Applications](#)
 - [Yield Example](#)
 - [Generator Expression](#)
 - [next\(\) Function](#)
 - [Fibonacci Generator](#)
 - [Infinite Generator](#)
 - [Summary](#)
-

Generators Overview



Generators are a special feature in Python that allow values to be produced **one at a time** instead of storing all values in memory at once.

Unlike lists, which generate and store every value immediately, generators create values only when they are needed. This makes them more memory-efficient and suitable for working with large datasets or continuous streams of data.

Think of a **vending machine**. When you press a button, it dispenses only the selected item instead of giving you every item inside the machine. Similarly, a generator produces one value whenever it is requested.

This behavior is known as **Lazy Evaluation**, where values are generated on demand rather than being created and stored upfront.

Understanding the Image

 Generators Working

The image above illustrates the main advantage of generators.

Step 1: Generator

A generator produces values **one at a time** instead of generating all values at once.

Unlike lists, it does not store every value in memory.

Step 2: Access Data in Pieces

Whenever a value is requested, the generator produces only that value.

This allows applications to process data incrementally instead of loading the entire dataset into memory.

Step 3: Compute Data in Pieces

After producing one value, the generator pauses its execution.

When the next value is requested, it resumes from the same point and computes only the next required value.

Step 4: Memory Savings

Since generators create values only when needed, they consume much less memory than lists.

This makes generators ideal for:

- Large datasets
- Reading large files
- Data streaming
- Processing continuous data
- Memory-efficient applications

The key idea shown in the image is that **generators process data in small pieces**, which results in significant **memory savings**.

Why Use Generators?

Generators are widely used because they provide several advantages over creating complete lists.

Memory Efficient

Generators produce values one at a time, reducing memory usage.

Lazy Evaluation

Values are generated only when required.

Faster Processing

Programs can start processing data immediately without waiting for all values to be created.

Suitable for Large Data

Generators work efficiently with very large datasets and continuous data streams.

Cleaner Code

The `yield` keyword makes it easier to generate sequences without manually managing iteration.

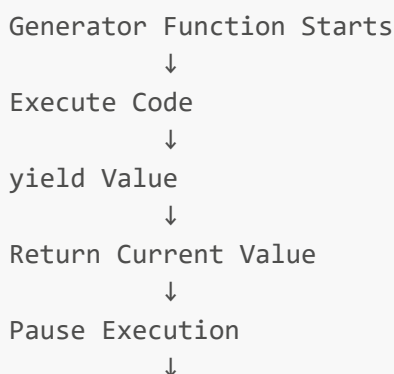
How Generators Work

A generator function uses the `yield` keyword instead of `return`.

When Python encounters a `yield` statement:

- It returns the current value.
- The function pauses execution.
- Python remembers the function's current state.
- When the next value is requested, execution resumes immediately after the previous `yield` statement.

Generator Workflow



```
Next Value Requested
    ↓
Resume After yield
    ↓
Generate Next Value
    ↓
Repeat Until Completed
```

Generator vs List

Both generators and lists are used to produce sequences of values. However, they differ in how they generate and store data.

Feature	List	Generator
Memory Usage	High	Low
Stores All Values	Yes	No
Lazy Evaluation	No	Yes
Suitable for Large Data	No	Yes
Creates Values	Immediately	On Demand
Syntax	<code>[]</code>	<code>yield</code>

Key Difference

A **list** creates and stores all values in memory before they are used.

A **generator** creates only one value at a time whenever it is requested, reducing memory consumption.

When to Use Generators?

Generators are useful when:

- Processing large datasets
- Reading large files
- Streaming data
- Processing log files
- Fetching database records
- Building data pipelines
- Working with infinite sequences

Whenever storing all values in memory is unnecessary, a generator is usually a better choice than a list.

Real-World Applications

Generators are commonly used in:

- Reading large files line by line
 - Processing live data streams
 - Generating mathematical sequences
 - Pagination systems
 - Log file analysis
 - Database record processing
 - Machine learning data pipelines
-

Examples

Concept	File
Yield Example	01_yield_example.py
Generator Expression	02_generator_expression.py
next() Function	03_next_example.py
Fibonacci Generator	04_fibonacci_generator.py
Infinite Generator	05_infinite_generator.py

1. Yield Example

Explanation

The **yield** keyword is used to create a generator function.

Unlike the **return** statement, which ends the function after returning a value, **yield** returns one value at a time and pauses the function's execution.

When the next value is requested, the generator resumes execution from the point where it previously stopped.

This allows generators to produce values on demand, making them more memory-efficient than lists.

Why Use **yield**?

The **yield** keyword is useful when:

- Values should be generated one at a time.
 - Working with large datasets.
 - Reducing memory usage.
 - Processing data efficiently.
 - Creating custom iterators.
-

Syntax

```
def generator_name():  
  
    yield value1  
  
    yield value2  
  
    yield value3
```

Syntax Explanation

- **def** defines a generator function.
- **yield** returns one value at a time.
- After each **yield**, the function pauses.
- The function resumes when the next value is requested.

Code Flow

```
Define Generator Function  
    ↓  
Call Generator Function  
    ↓  
Generator Object Created  
    ↓  
Execute Until First yield  
    ↓  
Return First Value  
    ↓  
Pause Execution  
    ↓  
Request Next Value  
    ↓  
Resume Execution  
    ↓  
Return Next Value  
    ↓  
Repeat Until Completed
```

How Python Executes

When Python sees:

```
def employee_ids():
```

```
yield 101
yield 102
yield 103
```

It creates a generator object instead of executing the function immediately.

When the generator is used in a loop:

```
for emp_id in employee_ids():
    print(emp_id)
```

Python executes the following steps:

```
Generator Starts
    ↓
yield 101
    ↓
Pause
    ↓
Resume
    ↓
yield 102
    ↓
Pause
    ↓
Resume
    ↓
yield 103
    ↓
Generator Ends
```

Difference Between `return` and `yield`

<code>return</code>	<code>yield</code>
Returns a value and ends the function	Returns a value and pauses the function
Function executes only once	Function resumes execution multiple times
Stores all values in memory	Generates values one at a time
Less suitable for large datasets	Ideal for large datasets

Example

Example: Generate employee IDs one by one instead of storing all employee IDs in memory.

Code

```
def employee_ids():  
  
    yield 101  
    yield 102  
    yield 103  
  
for emp_id in employee_ids():  
    print("Employee ID:", emp_id)
```

Code Explanation

Step 1

```
def employee_ids():
```

Defines a generator function.

Since it contains the `yield` keyword, Python treats it as a generator instead of a normal function.

Step 2

```
yield 101
```

Returns the first employee ID (101) and pauses the function.

Step 3

```
yield 102
```

When the next value is requested, execution resumes from this point and returns 102.

Step 4

```
yield 103
```

Execution resumes again and returns the final employee ID (103).

Step 5

```
for emp_id in employee_ids():
```

The `for` loop automatically requests one value at a time from the generator until all values have been generated.

Output

```
Employee ID: 101  
Employee ID: 102  
Employee ID: 103
```

 Yield Output

Key Takeaway

The `yield` keyword creates generator functions that produce one value at a time instead of storing all values in memory. This makes generators memory-efficient, scalable, and ideal for processing large datasets, streams of data, and custom iterators.

2. Generator Expression

Explanation

A Generator Expression is a concise way to create generators using a syntax similar to list comprehensions.

Unlike list comprehensions, which create and store all values in memory, generator expressions produce values one at a time only when they are needed.

This makes generator expressions more memory-efficient, especially when working with large datasets.

Why Use Generator Expressions?

Generator expressions are useful when:

- Creating generators in a single line.
- Reducing memory usage.
- Processing large datasets efficiently.
- Generating values only when required.

- Writing cleaner and shorter code.

Syntax

```
(expression for item in iterable)
```

Syntax Explanation

- **expression** → The value to generate.
- **item** → The current element from the iterable.
- **iterable** → The data source (such as **range()**, list, tuple, etc.).
- Parentheses **()** indicate a generator expression.

Code Flow

```
Create Generator Expression
      ↓
Generator Object Created
      ↓
Request Next Value
      ↓
Expression Evaluated
      ↓
Return Generated Value
      ↓
Pause Execution
      ↓
Request Next Value
      ↓
Generate Next Value
      ↓
Repeat Until Completed
```

How Python Executes

When Python sees:

```
squares = (x * x for x in range(1, 6))
```

it **does not calculate all square values immediately**.

Instead, it creates a generator object.

When the generator is used in a loop:

```
for num in squares:
    print(num)
```

Python performs the following steps:



Only one value is generated at a time.

Generator Expression vs List Comprehension

List Comprehension	Generator Expression
Uses []	Uses ()
Stores all values in memory	Generates values one at a time
Higher memory usage	Lower memory usage
Suitable for small datasets	Suitable for large datasets
Immediate evaluation	Lazy evaluation

Example

Example: Generate the squares of numbers one at a time without storing all values in memory.

Code

```
squares = (x * x for x in range(1, 6))

for num in squares:
    print(num)
```

Code Explanation

Step 1

```
squares = (x * x for x in range(1, 6))
```

Creates a generator expression.

No square values are calculated yet. Python simply creates a generator object.

Step 2

```
range(1, 6)
```

Generates the numbers:

```
1 2 3 4 5
```

Step 3

```
x * x
```

Calculates the square of each number only when it is requested.

For example:

```
1 × 1 = 1
2 × 2 = 4
3 × 3 = 9
...
```

Step 4

```
for num in squares:
```

The `for` loop requests one value at a time from the generator.

Each iteration produces the next square value until all values have been generated.

Output

```
1
4
9
16
25
```

Real-World Applications

Generator expressions are commonly used in:

- Processing large datasets
 - Reading large files
 - Data pipelines
 - Log file processing
 - Machine learning preprocessing
 - Streaming data
-

Key Takeaway

A Generator Expression provides a concise and memory-efficient way to create generators. Instead of storing all values in memory, it generates each value only when needed, making it ideal for processing large datasets and improving application performance.

3. next() Function

Explanation

The `next()` function is used to retrieve the next value from a generator.

Each time `next()` is called, the generator resumes execution from the previous `yield` statement, returns the next value, and pauses again until the next request.

Unlike a `for` loop, which automatically retrieves all values, the `next()` function gives developers manual control over generator execution.

Why Use `next()`?

The `next()` function is useful when:

- Retrieving generator values one at a time.
 - Controlling the execution of a generator manually.
 - Processing data only when needed.
 - Working with data streams.
 - Building custom iterators.
-

Syntax

```
generator = generator_function()

next(generator)
```

Syntax Explanation

- `generator_function()` creates a generator object.
 - `next(generator)` requests the next available value.
 - After returning a value, the generator pauses until `next()` is called again.
-

Code Flow

```
Create Generator
  ↓
Generator Object Created
  ↓
Call next()
  ↓
Execute Until First yield
  ↓
Return First Value
  ↓
Pause Execution
  ↓
Call next()
  ↓
Resume Execution
  ↓
Return Next Value
```

↓
Repeat Until Generator Ends

How Python Executes

When Python sees:

```
emp = employee_ids()
```

it creates a generator object.

When:

```
next(emp)
```

is called, Python starts executing the generator until it reaches the first **yield**.

Execution Flow:

```
Generator Starts
  ↓
yield 101
  ↓
Pause
  ↓
next(emp)
  ↓
yield 102
  ↓
Pause
  ↓
next(emp)
  ↓
yield 103
  ↓
Generator Ends
```

If **next()** is called again after all values have been generated, Python raises a **StopIteration** exception.

Example

Example: Generate employee IDs one at a time using the **next()** function.

Code

```
def employee_ids():  
  
    yield 101  
    yield 102  
    yield 103  
  
emp = employee_ids()  
  
print(next(emp))  
print(next(emp))  
print(next(emp))
```

Code Explanation

Step 1

```
def employee_ids():
```

Defines a generator function.

Step 2

```
    yield 101  
    yield 102  
    yield 103
```

Returns one employee ID at a time.

After each `yield`, the generator pauses until the next value is requested.

Step 3

```
emp = employee_ids()
```

Creates a generator object.

The generator is created, but no code inside the function executes yet.

Step 4

```
next(emp)
```

Starts executing the generator.

The first `yield` returns:

```
101
```

The generator then pauses.

Step 5

```
next(emp)
```

Execution resumes from the previous `yield` statement.

The next value returned is:

```
102
```

Step 6

```
next(emp)
```

The generator resumes again and returns:

```
103
```

After the final value is returned, the generator finishes execution.

Output

```
101  
102
```

Real-World Applications

The `next()` function is commonly used in:

- Reading large files line by line
 - Processing streaming data
 - Building custom iterators
 - Reading API responses
 - Processing database records
 - Lazy evaluation
-

Key Takeaway

The `next()` function retrieves one value at a time from a generator and gives developers complete control over when the next value is produced. This makes generators efficient for processing large datasets and streaming data without loading everything into memory.

4. Fibonacci Generator

Explanation



A Fibonacci Generator produces Fibonacci numbers one at a time using the `yield` keyword.

The Fibonacci sequence is a series of numbers where each number is the sum of the two preceding numbers.

```
0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...
```

Instead of storing the entire sequence in memory, a generator calculates and returns each Fibonacci number only when it is requested.

This makes generators an efficient way to generate long mathematical sequences.

Why Use a Fibonacci Generator?

A Fibonacci Generator is useful because it:

- Generates numbers on demand.
- Uses less memory.
- Produces long sequences efficiently.

- Demonstrates the practical use of generators.
-

Syntax

```
def fibonacci(n):  
    a, b = 0, 1  
  
    for _ in range(n):  
        yield a  
  
        a, b = b, a + b
```

Syntax Explanation

- `a` stores the current Fibonacci number.
 - `b` stores the next Fibonacci number.
 - `yield a` returns one Fibonacci number at a time.
 - `a, b = b, a + b` updates the sequence for the next iteration.
-

Code Flow

```
Initialize a = 0, b = 1  
    ↓  
Start Loop  
    ↓  
yield Current Value (a)  
    ↓  
Pause Execution  
    ↓  
Next Value Requested  
    ↓  
Update Values  
(a = b, b = a + b)  
    ↓  
Generate Next Fibonacci Number  
    ↓  
Repeat Until Loop Ends
```

How Python Executes

When Python executes:

```
for num in fibonacci(10):  
    print(num)
```

the following steps occur:

```
Generator Starts  
    ↓  
yield 0  
    ↓  
Pause  
    ↓  
Update Values  
    ↓  
yield 1  
    ↓  
Pause  
    ↓  
Update Values  
    ↓  
yield 1  
    ↓  
Pause  
    ↓  
Continue Until 10 Numbers Are Generated
```

The generator produces one Fibonacci number at a time and pauses after each **yield**.

Example

Example: Generate the first 10 Fibonacci numbers using a generator.

Code

```
def fibonacci(n):  
    a, b = 0, 1  
    for _ in range(n):  
        yield a  
        a, b = b, a + b
```

```
for num in fibonacci(10):  
    print(num)
```

Code Explanation

Step 1

```
def fibonacci(n):
```

Defines a generator function that produces `n` Fibonacci numbers.

Step 2

```
a, b = 0, 1
```

Initializes the first two Fibonacci numbers.

- `a` = Current number
 - `b` = Next number
-

Step 3

```
for _ in range(n):
```

Runs the loop `n` times to generate the required number of Fibonacci values.

Step 4

```
yield a
```

Returns the current Fibonacci number and pauses execution.

Step 5

```
a, b = b, a + b
```

Updates the variables to generate the next Fibonacci number.

For example:

```
a = 0, b = 1
  ↓
a = 1, b = 1
  ↓
a = 1, b = 2
  ↓
a = 2, b = 3
```

This process continues until all numbers have been generated.

Step 6

```
for num in fibonacci(10):
```

The `for` loop automatically requests one Fibonacci number at a time from the generator until the sequence is complete.

Output

```
0
1
1
2
3
5
8
13
21
34
```

Real-World Applications

Fibonacci Generators are commonly used in:

- Mathematical computations
- Algorithm design
- Dynamic programming
- Sequence generation
- Educational demonstrations of generators

Key Takeaway

A Fibonacci Generator uses the `yield` keyword to generate Fibonacci numbers one at a time instead of storing the entire sequence in memory. This approach is memory-efficient, scalable, and demonstrates how generators can be used to create mathematical sequences efficiently.

5. Infinite Generator

Explanation

An Infinite Generator continuously produces values without a predefined stopping condition.

Unlike normal generators that generate a fixed number of values, an infinite generator keeps producing values until the program stops requesting them.

Since values are generated one at a time using the `yield` keyword, infinite generators consume very little memory, making them suitable for continuous data generation.

Why Use an Infinite Generator?

Infinite generators are useful when:

- Generating unlimited values.
 - Processing continuous data streams.
 - Working with real-time applications.
 - Producing values only when needed.
 - Reducing memory consumption.
-

Syntax

```
def generator_name():  
    value = initial_value  
  
    while True:  
        yield value  
  
        value += 1
```

Syntax Explanation

- `while True` creates an infinite loop.
- `yield` returns one value at a time.

- The generator pauses after each `yield`.
- Execution resumes when the next value is requested.

Code Flow

```
Create Generator
  ↓
Initialize Value
  ↓
Start Infinite Loop
  ↓
yield Current Value
  ↓
Pause Execution
  ↓
Next Value Requested
  ↓
Increase Value
  ↓
yield Next Value
  ↓
Repeat Forever
```

How Python Executes

When Python executes:

```
numbers = infinite_numbers()
```

it creates a generator object.

When:

```
next(numbers)
```

is called, Python performs:

```
Generator Starts
  ↓
yield 1
  ↓
Pause
  ↓
Resume
```

↓
yield 2
↓
Pause
↓
Resume
↓
yield 3
↓
Continue Forever

The generator never stops unless the program stops requesting values.

Example

Example: Generate an infinite sequence of numbers using a generator.

Code

```
def infinite_numbers():  
  
    num = 1  
  
    while True:  
  
        yield num  
  
        num += 1  
  
numbers = infinite_numbers()  
  
for _ in range(5):  
    print(next(numbers))
```

Code Explanation

Step 1

```
def infinite_numbers():
```

Defines a generator function.

Step 2

```
num = 1
```

Initializes the starting value.

Step 3

```
while True:
```

Creates an infinite loop that continues generating values.

Step 4

```
yield num
```

Returns the current value and pauses execution.

Step 5

```
num += 1
```

Increments the value so the next number can be generated.

Step 6

```
numbers = infinite_numbers()
```

Creates a generator object.

The generator is ready to produce values but has not executed yet.

Step 7

```
next(numbers)
```

Requests the next value from the generator.

Each call resumes execution from the previous `yield` statement and returns the next number.

Step 8

```
for _ in range(5):
```

Requests five values from the generator.

Although the generator can produce values indefinitely, the loop limits the output to the first five numbers.

Output

```
1
2
3
4
5
```



Infinite Generator Output

Real-World Applications

Infinite generators are commonly used in:

- Live data streaming
 - Sensor data processing
 - Real-time analytics
 - Log monitoring systems
 - Continuous event generation
 - Simulation systems
-

Key Takeaway

An Infinite Generator produces values continuously without a predefined ending condition. By generating one value at a time using the `yield` keyword, it remains memory-efficient and is ideal for handling continuous or real-time data.

Generators Summary

Generators are special functions that use the `yield` keyword to produce values one at a time instead of returning all values at once.

Unlike lists, generators generate values only when they are requested, making them memory-efficient and suitable for processing large datasets, continuous data streams, and mathematical sequences.

Throughout this module, we explored the following concepts:

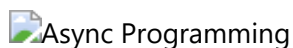
- Yield Example
- Generator Expression
- `next()` Function
- Fibonacci Generator
- Infinite Generator

By understanding these concepts, you can efficiently generate data on demand, reduce memory usage, and build scalable Python applications. Generators are widely used in file processing, streaming data, database operations, machine learning pipelines, and many other real-world applications where performance and memory optimization are important.

Async Programming

- [Async Programming Overview](#)
- [Why Async Programming?](#)
- [Synchronous vs Asynchronous](#)
- [Real-World Example](#)
- [How Async Works](#)
- [Async & Await](#)
- [Tasks](#)
- [`asyncio.gather\(\)`](#)
- [Async API Call](#)
- [Summary](#)

Async Programming Overview



Async Programming is a programming technique that allows multiple tasks to run efficiently without blocking the execution of a program.

In traditional synchronous programming, tasks are executed one after another. If one task takes a long time to complete, the entire program must wait before moving to the next task.

Async Programming solves this problem by allowing the program to continue executing other tasks while waiting for slow operations such as API requests, database queries, file operations, or network communication.

Python provides built-in support for asynchronous programming through the `asyncio` module using the `async` and `await` keywords.

Asynchronous Programming

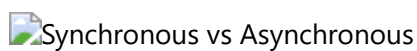
In asynchronous execution, a task can pause while waiting, allowing other tasks to continue running.

```
Task 1 Starts
  ↓
Waiting...
  ↓
Task 2 Executes
  ↓
Task 3 Executes
  ↓
Task 1 Resumes
  ↓
Task Completes
```

Instead of wasting time while waiting, Python efficiently switches to other available tasks.

This improves resource utilization and application performance.

Synchronous vs Asynchronous



The image above compares how **Synchronous** and **Asynchronous** programming handle multiple requests.

Synchronous Programming

In synchronous programming, tasks are executed **one after another**.

A new task cannot start until the previous task has completely finished.

For example:

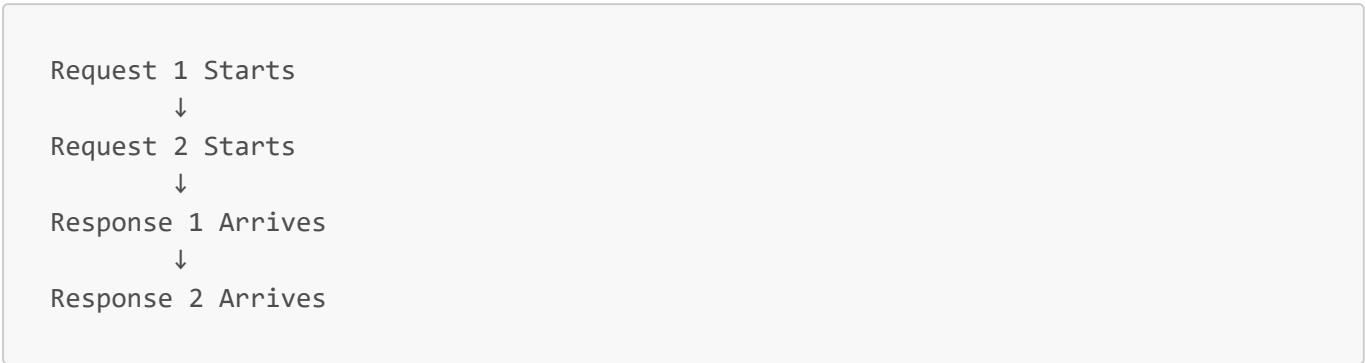
```
Request 1
  ↓
Wait for Response 1
  ↓
Request 2
  ↓
Wait for Response 2
  ↓
Program Continues
```

If **Request 1** takes a long time, **Request 2** must wait, which increases the overall execution time.

Asynchronous Programming

In asynchronous programming, multiple requests can be started without waiting for previous requests to finish.

For example:



While **Request 1** is waiting for a response, the program can continue executing **Request 2** or perform other useful work.

This reduces waiting time and improves application performance.

Comparison

Synchronous	Asynchronous
Tasks execute one after another	Multiple tasks can execute concurrently
Each task waits for the previous one to finish	Tasks continue while waiting for other operations
Blocking execution	Non-blocking execution
Slower for I/O-bound operations	Faster for I/O-bound operations
CPU remains idle during waiting	CPU performs other useful work while waiting
Simpler to understand	Better performance for modern applications

When to Use Each Approach

Use Synchronous Programming

- Simple scripts
- Small applications
- Sequential calculations
- Tasks with no waiting time

Use Asynchronous Programming

- API requests
- Web applications
- Database operations
- File handling

- Network communication
 - Real-time applications
 - Chat applications
 - Streaming services
-

1. Async & Await

Explanation

Async Programming allows a program to perform multiple operations without blocking execution.

Python uses the `async` and `await` keywords together with the `asyncio` module to implement asynchronous programming.

- `async` is used to define a coroutine (asynchronous function).
- `await` pauses the current coroutine until an asynchronous operation completes.
- While one coroutine is waiting, Python can execute other coroutines.

This makes asynchronous programming ideal for I/O-bound tasks such as API requests, database queries, file operations, and network communication.

Why Use Async & Await?

Async & Await are useful when:

- Running multiple tasks efficiently.
 - Preventing blocking operations.
 - Improving application performance.
 - Handling API and network requests.
 - Working with databases and files.
 - Building responsive applications.
-

Syntax

```
import asyncio

async def function_name():

    await asyncio.sleep(1)

asyncio.run(function_name())
```

Syntax Explanation

- `async` defines an asynchronous function.

- `await` pauses the coroutine until the awaited task completes.
 - `asyncio.run()` starts the event loop and executes the coroutine.
-

Code Flow

```
Define Coroutine
  ↓
Start Event Loop
  ↓
Execute Coroutine
  ↓
Encounter await
  ↓
Pause Current Coroutine
  ↓
Execute Other Tasks
  ↓
Resume Coroutine
  ↓
Coroutine Completes
```

How Python Executes

When Python executes:

```
asyncio.run(greet())
```

the following steps occur:

```
Event Loop Starts
  ↓
greet() Starts
  ↓
Print "Hello"
  ↓
await asyncio.sleep(2)
  ↓
Coroutine Pauses
  ↓
Other Tasks Can Execute
  ↓
2 Seconds Complete
  ↓
Coroutine Resumes
  ↓
Print "Welcome to Async Programming"
```

↓
Coroutine Ends
↓
Event Loop Stops

The `await` statement pauses only the current coroutine, not the entire program.

Example

Example: A food delivery application can continue processing other customer requests while waiting for an order to be prepared.

Code

```
import asyncio

async def greet():

    print("Hello")

    await asyncio.sleep(2)

    print("Welcome to Async Programming")

asyncio.run(greet())
```

Code Explanation

Step 1

```
import asyncio
```

Imports Python's built-in `asyncio` module, which provides support for asynchronous programming.

Step 2

```
async def greet():
```

Defines a coroutine using the `async` keyword.

Unlike a normal function, a coroutine can pause and resume its execution.

Step 3

```
print("Hello")
```

Executes immediately when the coroutine starts.

Output:

```
Hello
```

Step 4

```
await asyncio.sleep(2)
```

Pauses the current coroutine for two seconds.

During this waiting period, Python can execute other asynchronous tasks.

Step 5

```
print("Welcome to Async Programming")
```

After the waiting period is complete, the coroutine resumes execution and prints the message.

Step 6

```
asyncio.run(greet())
```

Creates an event loop and starts executing the coroutine.

The event loop manages the execution of asynchronous tasks until they are completed.

Output

```
Hello
(Waits for 2 seconds)
Welcome to Async Programming
```

 output

Real-World Applications

Async Programming is commonly used in:

- API Requests
- Database Operations
- Web Applications
- File Processing
- Chat Applications
- Real-Time Notifications
- Network Communication
- Web Scraping

Key Takeaway

The `async` keyword creates a coroutine, while the `await` keyword pauses the current coroutine without blocking the entire program. This allows Python to perform other tasks during waiting periods, making applications faster, more responsive, and more efficient for I/O-bound operations.

2. Tasks

Explanation

A **Task** is used to schedule and execute a coroutine concurrently.

Normally, when one coroutine is waiting, the program would pause before executing the next coroutine. By creating tasks, multiple coroutines can run concurrently, allowing the program to perform other work while waiting for slow operations.

Python provides the `asyncio.create_task()` function to create and manage tasks.

Why Use Tasks?

Tasks are useful when:

- Running multiple coroutines concurrently.
- Improving application responsiveness.
- Reducing waiting time for I/O operations.
- Making better use of system resources.

- Building scalable asynchronous applications.
-

Syntax

```
task = asyncio.create_task(coroutine())
```

Syntax Explanation

- `asyncio.create_task()` schedules a coroutine to run concurrently.
 - The returned task object represents the running coroutine.
 - `await task` waits until the task completes.
-

Code Flow

```
Start Event Loop
    ↓
Create Task 1
    ↓
Create Task 2
    ↓
Both Tasks Start
    ↓
Task 1 Waiting
    ↓
Task 2 Executes
    ↓
Task 1 Resumes
    ↓
Both Tasks Complete
```

How Python Executes

When Python executes:

```
task1 = asyncio.create_task(prepare_order(1))
task2 = asyncio.create_task(prepare_order(2))
```

Python immediately schedules both coroutines.

Execution Flow:

Event Loop Starts



Task 1 Created



Task 2 Created



Task 1 Starts



Task 2 Starts



Task 1 Waiting



Task 2 Waiting



Task 1 Resumes



Task 2 Resumes



Both Tasks Finish

Unlike synchronous programming, the second task does not wait for the first task to finish before starting.

Example

Example: A food delivery application prepares multiple customer orders simultaneously instead of processing them one by one.

Code

```
import asyncio

async def prepare_order(order):

    print(f"Preparing Order {order}")

    await asyncio.sleep(2)

    print(f"Order {order} Ready")

async def main():

    task1 = asyncio.create_task(
        prepare_order(1)
    )

    task2 = asyncio.create_task(
```

```
        prepare_order(2)
    )

    await task1
    await task2

asyncio.run(main())
```

Code Explanation

Step 1

```
import asyncio
```

Imports Python's built-in `asyncio` module.

Step 2

```
async def prepare_order(order):
```

Defines a coroutine that prepares a customer order.

Step 3

```
    await asyncio.sleep(2)
```

Simulates the time required to prepare the order.

While waiting, Python can execute other tasks.

Step 4

```
    task1 = asyncio.create_task(
        prepare_order(1)
    )
```

Creates the first asynchronous task.

The task is scheduled immediately by the event loop.

Step 5

```
task2 = asyncio.create_task(  
    prepare_order(2)  
)
```

Creates the second asynchronous task.

Both tasks can now run concurrently.

Step 6

```
await task1  
await task2
```

Waits until both tasks finish execution.

Step 7

```
asyncio.run(main())
```

Starts the event loop and executes the `main()` coroutine.

Output

```
Preparing Order 1  
Preparing Order 2  
Order 1 Ready  
Order 2 Ready
```



Real-World Applications

Tasks are commonly used in:

- Food Delivery Systems
- Chat Applications
- Web Servers
- API Requests

- Database Operations
 - Background Processing
 - File Downloads
 - Email Services
-

Key Takeaway

`asyncio.create_task()` allows multiple coroutines to run concurrently without blocking one another. This improves application responsiveness and makes asynchronous programs more efficient for handling multiple I/O-bound operations simultaneously.

3. asyncio.gather()

Explanation

The `asyncio.gather()` function is used to execute multiple coroutines concurrently and wait until all of them complete.

Instead of creating and awaiting multiple tasks separately, `asyncio.gather()` groups multiple coroutines together and runs them concurrently.

It is useful when several independent asynchronous operations need to complete before the program continues.

Why Use `asyncio.gather()`?

`asyncio.gather()` is useful when:

- Running multiple coroutines at the same time.
 - Waiting for all coroutines to finish.
 - Simplifying concurrent task management.
 - Reducing overall execution time.
 - Collecting results from multiple coroutines.
-

Syntax

```
await asyncio.gather(  
    coroutine1(),  
    coroutine2(),  
    coroutine3()  
)
```

Syntax Explanation

- `asyncio.gather()` groups multiple coroutines.
- Each coroutine starts executing concurrently.
- `await` pauses until all coroutines complete.
- After completion, execution continues.

Code Flow

```
Start Event Loop
    ↓
Create Coroutine 1
    ↓
Create Coroutine 2
    ↓
Create Coroutine 3
    ↓
asyncio.gather()
    ↓
All Coroutines Execute Concurrently
    ↓
All Coroutines Complete
    ↓
Program Continues
```

How Python Executes

When Python executes:

```
await asyncio.gather(
    prepare_order(1),
    prepare_order(2),
    prepare_order(3)
)
```

the following steps occur:

```
Event Loop Starts
    ↓
Coroutine 1 Starts
    ↓
Coroutine 2 Starts
    ↓
Coroutine 3 Starts
    ↓
All Coroutines Run Concurrently
    ↓
```

```
Coroutine 1 Completes
    ↓
Coroutine 2 Completes
    ↓
Coroutine 3 Completes
    ↓
gather() Returns
```

The program waits until every coroutine finishes before moving to the next statement.

Example

Example: A food delivery application prepares multiple customer orders simultaneously and waits until every order is ready before delivering them.

Code

```
import asyncio

async def prepare_order(order):

    print(f"Preparing Order {order}")

    await asyncio.sleep(2)

    print(f"Order {order} Ready")

async def main():

    await asyncio.gather(
        prepare_order(1),
        prepare_order(2),
        prepare_order(3)
    )

asyncio.run(main())
```

Code Explanation

Step 1

```
import asyncio
```

Imports Python's built-in `asyncio` module.

Step 2

```
async def prepare_order(order):
```

Defines a coroutine that prepares a customer order.

Step 3

```
await asyncio.sleep(2)
```

Simulates the time required to prepare an order.

During this waiting period, other coroutines continue executing.

Step 4

```
await asyncio.gather(  
    prepare_order(1),  
    prepare_order(2),  
    prepare_order(3)  
)
```

Groups three coroutines together and starts them concurrently.

Instead of waiting for one order to finish before starting the next, all three begin execution almost at the same time.

Step 5

```
asyncio.run(main())
```

Starts the event loop and executes the `main()` coroutine.

The program waits until every coroutine inside `asyncio.gather()` has completed.

Output

```
Preparing Order 1
Preparing Order 2
Preparing Order 3
Order 1 Ready
Order 2 Ready
Order 3 Ready
```



Real-World Applications

`asyncio.gather()` is commonly used in:

- Multiple API Requests
- Database Queries
- Web Scraping
- Parallel File Processing
- Microservices Communication
- Downloading Multiple Files
- Sending Multiple Emails
- Running Independent Background Tasks

Key Takeaway

`asyncio.gather()` executes multiple coroutines concurrently and waits until all of them complete. It simplifies asynchronous programming, improves performance, and is ideal when several independent tasks need to finish before the program continues.

4. Async API Call

Explanation

One of the most common applications of Async Programming is making API requests.

In synchronous programming, the application waits for each API request to finish before sending the next request. This increases the overall execution time.

With asynchronous programming, multiple API requests can be sent concurrently. While one request is waiting for a response, other requests can continue executing.

This improves application performance and responsiveness, especially when communicating with web services, cloud platforms, and microservices.

Why Use Async API Calls?

Async API Calls are useful when:

- Sending multiple API requests concurrently.
 - Reducing waiting time for network operations.
 - Improving application responsiveness.
 - Efficiently handling slow external services.
 - Building scalable web applications.
-

Syntax

```
import asyncio

async def fetch_data():

    await asyncio.sleep(2)

    return "Data"

async def main():

    result = await fetch_data()

    print(result)

asyncio.run(main())
```

Syntax Explanation

- `async` defines a coroutine.
 - `await` waits for the API response without blocking the program.
 - `asyncio.run()` starts the event loop and executes the coroutine.
-

Code Flow

```
Start Event Loop
  ↓
Create API Tasks
  ↓
Send API Requests
  ↓
Waiting for Responses
  ↓
Execute Other Tasks
```

```
↓  
Receive API Responses  
↓  
Process Data  
↓  
Program Completes
```

How Python Executes

When Python executes:

```
user = asyncio.create_task(fetch_user())  
orders = asyncio.create_task(fetch_orders())
```

the following steps occur:

```
Event Loop Starts  
↓  
Send User Request  
↓  
Send Order Request  
↓  
Both Requests Wait  
↓  
Other Tasks Can Execute  
↓  
User Response Received  
↓  
Order Response Received  
↓  
Display Results
```

Instead of waiting for one request to finish before starting the next, both requests execute concurrently.

Example

Example: Fetch user information and order details simultaneously using asynchronous API calls.

Code

```
import asyncio  
  
async def fetch_user():
```

```
    print("Fetching User Data...")

    await asyncio.sleep(2)

    return "User Data"

async def fetch_orders():

    print("Fetching Orders...")

    await asyncio.sleep(3)

    return "Order Data"

async def main():

    user = asyncio.create_task(fetch_user())

    orders = asyncio.create_task(fetch_orders())

    print("Waiting for API responses...")

    user_data = await user

    order_data = await orders

    print(user_data)

    print(order_data)

asyncio.run(main())
```

Code Explanation

Step 1

```
import asyncio
```

Imports Python's built-in `asyncio` module for asynchronous programming.

Step 2

```
async def fetch_user():
```

Defines a coroutine that simulates fetching user information from an API.

Step 3

```
await asyncio.sleep(2)
```

Simulates the time taken to receive the API response.

While waiting, Python can execute other asynchronous tasks.

Step 4

```
async def fetch_orders():
```

Defines another coroutine that simulates fetching order information.

Step 5

```
user = asyncio.create_task(fetch_user())
```

Creates the first asynchronous task.

The API request starts immediately.

Step 6

```
orders = asyncio.create_task(fetch_orders())
```

Creates the second asynchronous task.

Both API requests now execute concurrently.

Step 7

```
user_data = await user
order_data = await orders
```

Waits until both API requests complete and stores their returned data.

Step 8

```
asyncio.run(main())
```

Starts the event loop and executes the `main()` coroutine.

Output

```
Waiting for API responses...
Fetching User Data...
Fetching Orders...
User Data
Order Data
```



output

Real-World Applications

Async API Calls are commonly used in:

- Weather Applications
 - Payment Gateways
 - Social Media Platforms
 - E-Commerce Systems
 - Cloud Services
 - Microservices Architecture
 - Stock Market Applications
 - Online Booking Systems
-

Key Takeaway

Async API Calls allow applications to communicate with external services without blocking execution. By handling multiple requests concurrently, they improve responsiveness, reduce waiting time, and make modern web applications more efficient.

Async Programming Summary

Async Programming enables applications to perform multiple operations efficiently without blocking execution.

By using the `asyncio` module along with the `async` and `await` keywords, Python can execute multiple tasks concurrently while waiting for I/O-bound operations such as API requests, database queries, file handling, and network communication.

Throughout this module, we explored the following concepts:

- Async & Await
- Tasks
- `asyncio.gather()`
- Async API Calls

These concepts form the foundation of asynchronous programming in Python and are widely used in modern web applications, cloud services, real-time systems, microservices, and high-performance applications. Understanding asynchronous programming helps developers build faster, more responsive, and scalable software.

Logging

- [Logging Overview](#)
 - [Why Use Logging?](#)
 - [How Logging Works](#)
 - [Logging Levels](#)
 - [Basic Example](#)
 - [Components of Logging](#)
 - [Real-World Example](#)
 - [Basic Logging](#)
 - [File Logging](#)
 - [Custom Formatter](#)
 - [Exception Logging](#)
 - [Summary](#)
-

Logging Overview



Logging is the process of recording events that occur while a program is running.

Instead of using multiple `print()` statements, Python provides the built-in `logging` module to record application activities, monitor errors, debug issues, and maintain execution history.

Logging helps developers understand application behavior, identify problems, and monitor software more effectively.

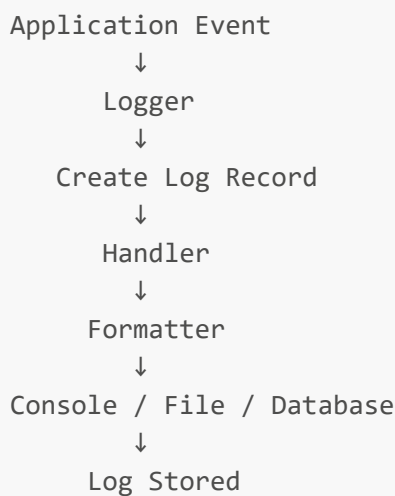
Understanding the Logging Process

When a program is running, many events occur such as a user logging in, a file being uploaded, or an error happening.

Instead of displaying these events using `print()`, Python's **logging** module records them in a structured way.

Every log message passes through several components before it is displayed on the screen or stored in a file.

The complete logging process is shown below.



Let's understand each step in detail.

Step 1: Application Event

Everything begins with an **event** inside the application.

An event is simply something that happens while the program is running.

Examples:

- User logs into the application.
- Customer places an order.
- File is uploaded.
- API request is received.
- Payment is completed.
- An error occurs.

For example:

```
User Login
```

At this point, Python knows that something important has happened, but it has not yet created a log message.

Step 2: Logger

The **Logger** receives information about the event.

It acts like a manager that decides whether the event should be logged.

For example:

```
logger.info("User logged in")
```

Here,

- `logger` receives the event.
- `info()` specifies the logging level.
- `"User logged in"` is the log message.

The logger then prepares this information for the next step.

Step 3: Create Log Record

After receiving the event, Python creates a **Log Record**.

A Log Record is a package that contains all the information about the event.

It may include:

- Log Message
- Date and Time
- Log Level
- File Name
- Function Name
- Line Number

For example, Python internally prepares something similar to:

```
Time       : 2026-06-22 10:30:15
Level      : INFO
Message    : User logged in
File       : app.py
Function   : login()
Line      : 25
```

This information is now ready to be displayed or stored.

Step 4: Handler

The **Handler** decides **where the log message should go**.

Think of the Handler as a delivery person.

It receives the log record and delivers it to the required destination.

Common destinations are:

- Console
- Log File
- Database
- Email
- Remote Logging Server

For example:

```
Logger
  ↓
Handler
  ↓
Console
```

or

```
Logger
  ↓
Handler
  ↓
application.log
```

Different handlers can be used for different purposes.

Step 5: Formatter

Before displaying or storing the log, Python uses a **Formatter**.

The Formatter decides **how the log message should look**.

Without a formatter, the message may appear in a simple format.

With a formatter, extra information can be added.

For example:

```
2026-06-22 10:30:15 - INFO - User logged in
```

A formatter can include:

- Date and Time
- Log Level
- File Name
- Function Name
- Line Number
- Log Message

This makes logs much easier to read and debug.

Step 6: Log Storage

Finally, the formatted log message reaches its destination.

Depending on the handler, the log may be:

- Displayed in the console.
- Saved in a log file.
- Stored in a database.
- Sent to an email.
- Uploaded to a monitoring server.

Example:

```
2026-06-22 10:30:15 - INFO - User logged in
```

This log can later be reviewed by developers to monitor the application, debug problems, and understand what happened during program execution.

Complete Logging Flow

```
User Login
  ↓
Logger Receives Event
  ↓
Python Creates Log Record
  ↓
Handler Selects Destination
  ↓
Formatter Formats Message
  ↓
Console / File / Database
```

↓
Log Stored Successfully

Real-World Example

Consider an **Online Shopping Application**.

Whenever a customer places an order, the application records important events such as:

- User Login
- Product Search
- Order Creation
- Payment Processing
- Order Confirmation
- Error Messages
- System Warnings

Instead of displaying these details on the screen, the application stores them in log files so developers can monitor and analyze the application's behavior later.

Real-World Applications

Logging is commonly used in:

- Web Applications
- Banking Systems
- E-Commerce Platforms
- Cloud Services
- API Monitoring
- Database Applications
- Security Auditing
- Error Tracking Systems

Examples

Concept	File
Basic Logging	01_basic_logging.py
File Logging	02_file_logging.py
Custom Formatter	03_custom_formatter.py
Exception Logging	04_exception_logging.py

1. Basic Logging

Explanation

Basic Logging is the simplest way to record messages generated while a program is running.

Instead of using multiple `print()` statements, Python provides the built-in `logging` module to record messages with different severity levels.

These log messages help developers monitor program execution, identify errors, and debug applications more effectively.

Why Use Basic Logging?

Basic Logging is useful when:

- Tracking program execution.
 - Debugging application issues.
 - Recording important events.
 - Displaying warning and error messages.
 - Monitoring application behavior.
-

Syntax

```
import logging

logging.basicConfig(level=logging.INFO)

logging.info("Message")
```

Syntax Explanation

- `import logging` imports Python's built-in logging module.
 - `logging.basicConfig()` configures the logging system.
 - `level=` specifies the minimum logging level to display.
 - `logging.info()` records an informational message.
-

Code Flow

```
Import logging Module
    ↓
Configure Logging
    ↓
Set Logging Level
    ↓
Generate Log Message
    ↓
Logging Module Processes Message
```

↓
Display Log on Console

How Python Executes

When Python executes:

```
logging.info("Application Started")
```

the following steps occur:

```
Program Starts
  ↓
Logging Module Initialized
  ↓
Logging Level Checked
  ↓
Create Log Record
  ↓
Format Message
  ↓
Display Message
```

Only messages that meet the configured logging level are displayed.

Example

Example: An e-commerce application records debugging information, user activities, warnings, and errors while processing customer orders.

Code

01_basic_logging.py

```
import logging

logging.basicConfig(
    level=logging.DEBUG
)

logging.debug("Debug Message")
logging.info("Info Message")
logging.warning("Warning Message")
```

```
logging.error("Error Message")
logging.critical("Critical Message")
```

Code Explanation

Step 1

```
import logging
```

Imports Python's built-in `logging` module.

Step 2

```
logging.basicConfig(
    level=logging.DEBUG
)
```

Configures the logging system.

The logging level is set to **DEBUG**, which means every log message from **DEBUG** and above will be displayed.

Step 3

```
logging.debug("Debug Message")
```

Displays a debugging message.

This is mainly used by developers while testing and debugging applications.

Step 4

```
logging.info("Info Message")
```

Displays general information about the application's execution.

Step 5

```
logging.warning("Warning Message")
```

Displays a warning indicating a possible problem that does not stop the application.

Step 6

```
logging.error("Error Message")
```

Displays an error message when an operation fails.

Step 7

```
logging.critical("Critical Message")
```

Displays a critical message for serious system failures that require immediate attention.

Output

```
DEBUG:root:Debug Message  
INFO:root:Info Message  
WARNING:root:Warning Message  
ERROR:root:Error Message  
CRITICAL:root:Critical Message
```

 Basic Logging Output

Understanding the Output

Each log message consists of three parts:

```
INFO:root:Info Message  
|      |      |  
|      |      | Log Message  
|      |      |  
|      |      | Logger Name  
|      |      |  
|      |      | Log Level
```

For example:

```
ERROR:root:Database Connection Failed
```

- **ERROR** → Log Level
- **root** → Default Logger Name
- **Database Connection Failed** → Log Message

Logging Levels

Level	Purpose
DEBUG	Displays detailed information for debugging.
INFO	Displays general information about program execution.
WARNING	Indicates a potential issue that does not stop the program.
ERROR	Reports an error that affects part of the application.
CRITICAL	Reports a serious system failure requiring immediate attention.

Key Takeaway

Basic Logging provides a structured way to record application events instead of using multiple `print()` statements. By using different logging levels, developers can easily monitor program execution, identify issues, and debug applications more efficiently.

2. File Logging

Explanation

File Logging stores log messages in a file instead of displaying them only in the console.

In real-world applications, developers often need to review application activities even after the program has finished executing. Saving logs in a file makes it easier to track application behavior, identify errors, and troubleshoot issues.

Python's `logging` module provides the `filename` parameter in the `basicConfig()` function to save log messages directly into a log file.

Why Use File Logging?

File Logging is useful when:

- Storing application logs permanently.
- Tracking application behavior over time.
- Debugging errors after program execution.

- Maintaining audit records.
 - Monitoring production applications.
-

Syntax

```
import logging

logging.basicConfig(
    filename="app.log",
    level=logging.INFO
)

logging.info("Message")
```

Syntax Explanation

- `filename="app.log"` specifies the file where log messages will be stored.
 - `level=logging.INFO` records log messages with the **INFO** level and above.
 - `logging.info()` writes a log message to the specified file.
-

Code Flow

```
Import logging Module
    ↓
Configure File Logging
    ↓
Specify Log File Name
    ↓
Generate Log Messages
    ↓
Logging Module Processes Messages
    ↓
Write Messages to File
```

How Python Executes

When Python executes:

```
logging.info("Application Started")
```

the following steps occur:

```
Program Starts
  ↓
Logging Module Initialized
  ↓
Open/Create app.log
  ↓
Create Log Record
  ↓
Format Log Message
  ↓
Write Message to app.log
```

If the log file does not already exist, Python automatically creates it.

Example

Example: A banking application stores transaction logs in a file so that administrators can review activities and troubleshoot issues later.

Code

02_file_logging.py

```
import logging

logging.basicConfig(
    filename="app.log",
    level=logging.INFO
)

logging.info("Application Started")
logging.warning("Low Balance Warning")
logging.error("Transaction Failed")

print("Logs saved to app.log")
```

Code Explanation

Step 1

```
import logging
```

Imports Python's built-in `logging` module.

Step 2

```
logging.basicConfig(  
    filename="app.log",  
    level=logging.INFO  
)
```

Configures the logging system.

- `filename="app.log"` stores log messages in the file named **app.log**.
 - `level=logging.INFO` records all log messages with **INFO**, **WARNING**, **ERROR**, and **CRITICAL** levels.
-

Step 3

```
logging.info("Application Started")
```

Writes an informational message to the log file.

Step 4

```
logging.warning("Low Balance Warning")
```

Writes a warning message indicating a potential issue.

Step 5

```
logging.error("Transaction Failed")
```

Writes an error message when an operation fails.

Step 6

```
print("Logs saved to app.log")
```

Displays a confirmation message in the console.

This message is **not** written to the log file because it uses the `print()` function.

Output

Console Output

```
Logs saved to app.log
```

app.log File Content

```
INFO:root:Application Started
WARNING:root:Low Balance Warning
ERROR:root:Transaction Failed
```

output1

output1

Understanding the Log File

Each line in the log file contains three parts:

```
INFO:root:Application Started
|         |         |
|         |         └─ Log Message
|         └─────────── Logger Name
└──────────────────── Log Level
```

For example:

```
ERROR:root:Transaction Failed
```

- **ERROR** → Log Level
- **root** → Default Logger Name
- **Transaction Failed** → Log Message

Benefits of File Logging

- Maintains application history.
- Makes debugging easier.
- Supports auditing and compliance.
- Preserves logs after the program exits.
- Useful for monitoring production systems.

Key Takeaway

File Logging stores application events permanently in a log file instead of displaying them only in the console. This allows developers to review application behavior, diagnose problems, and monitor software even after the program has finished running.

3. Custom Formatter

Explanation

A Custom Formatter allows developers to control how log messages are displayed.

By default, Python displays log messages in a simple format. However, in real-world applications, developers often need additional information such as the date, time, log level, filename, function name, or line number.

Using a custom formatter makes log messages more informative, easier to read, and simpler to debug.

Why Use a Custom Formatter?

A Custom Formatter is useful when:

- Displaying timestamps with log messages.
 - Identifying the log level.
 - Knowing which file generated the log.
 - Identifying the function where the log was created.
 - Making log files easier to analyze.
-

Syntax

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - %(message)s"
)

logging.info("Message")
```

Syntax Explanation

- `level=logging.INFO` displays log messages from the **INFO** level and above.
- `format=` specifies how the log message should appear.
- `%(asctime)s` displays the current date and time.
- `%(levelname)s` displays the logging level.

- `%(message)s` displays the log message.
-

Code Flow

```
Import logging Module
  ↓
Configure Logging
  ↓
Define Custom Format
  ↓
Generate Log Message
  ↓
Logging Module Applies Formatter
  ↓
Formatted Log Displayed
```

How Python Executes

When Python executes:

```
logging.info("Application Started")
```

the following steps occur:

```
Program Starts
  ↓
Logging Module Initialized
  ↓
Create Log Record
  ↓
Apply Custom Formatter
  ↓
Display Formatted Message
```

Instead of displaying only the message, Python formats it according to the pattern specified in the `format` parameter.

Example

Example: A web application records the timestamp, log level, and message whenever a user sends a request.

Code

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - %(message)s"
)

logging.info("Application Started")
logging.warning("Low Memory Warning")
logging.error("Database Connection Failed")
```

Code Explanation

Step 1

```
import logging
```

Imports Python's built-in `logging` module.

Step 2

```
logging.basicConfig(
```

Configures the logging system.

Step 3

```
level=logging.INFO
```

Specifies that log messages with the **INFO** level and above will be displayed.

Step 4

```
format="%(asctime)s - %(levelname)s - %(message)s"
```

Defines the format of each log message.

It contains:

- `%(asctime)s` → Displays the current date and time.
 - `%(levelname)s` → Displays the logging level.
 - `%(message)s` → Displays the actual log message.
-

Step 5

```
logging.info("Application Started")
```

Creates an informational log message.

The formatter automatically adds the timestamp and log level.

Step 6

```
logging.warning("Low Memory Warning")
```

Creates a warning log message.

Step 7

```
logging.error("Database Connection Failed")
```

Creates an error log message.

The formatter applies the same format to every log message.

Output

```
2026-06-20 10:30:15,123 - INFO - Application Started
2026-06-20 10:30:15,124 - WARNING - Low Memory Warning
2026-06-20 10:30:15,125 - ERROR - Database Connection Failed
```

 output

Understanding the Output

Each formatted log message contains multiple pieces of information.

```
2026-06-20 10:30:15,123 - INFO - Application Started
```

Log Message
Log Level
Date and Time

This additional information helps developers identify **when**, **where**, and **what** happened during program execution.

Common Formatting Options

Placeholder	Description
<code>%(asctime)s</code>	Displays the current date and time
<code>%(levelname)s</code>	Displays the logging level
<code>%(message)s</code>	Displays the log message
<code>%(filename)s</code>	Displays the file name
<code>%(funcName)s</code>	Displays the function name
<code>%(lineno)d</code>	Displays the line number
<code>%(name)s</code>	Displays the logger name
<code>%(module)s</code>	Displays the module name

Key Takeaway

A Custom Formatter allows developers to control how log messages are displayed by including useful information such as timestamps, log levels, filenames, function names, and messages. Well-formatted logs make debugging, monitoring, and maintaining applications much easier.

4. Exception Logging

Explanation

Exception Logging is used to record errors and exceptions that occur while a program is running.

Instead of allowing the program to stop without any information, Python records the error message along with detailed information about where and why the error occurred.

Python provides the `logging.exception()` method, which automatically logs the exception message together with the complete traceback. This makes it much easier for developers to identify and fix problems.

Why Use Exception Logging?

Exception Logging is useful when:

- Recording unexpected errors.
- Capturing complete traceback information.
- Simplifying debugging.
- Monitoring application failures.
- Improving application reliability.

Syntax

```
import logging

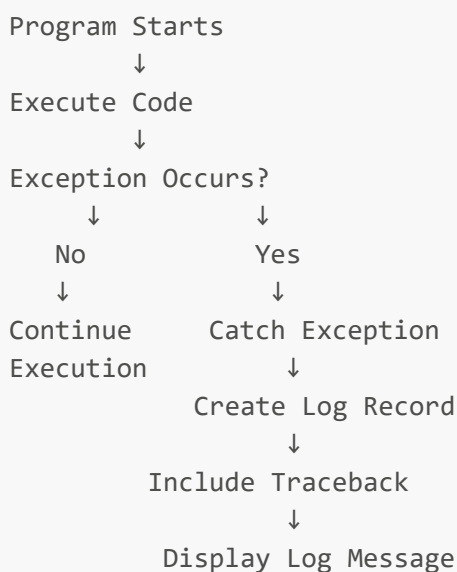
try:
    # Code that may cause an exception

except Exception:
    logging.exception("Error Message")
```

Syntax Explanation

- `try` contains the code that may raise an exception.
- `except` catches the exception.
- `logging.exception()` records the error message along with the traceback.
- The traceback helps identify where the exception occurred.

Code Flow



How Python Executes

When Python executes:

```
result = 10 / 0
```

the following steps occur:

```
Program Starts
  ↓
Execute Division
  ↓
ZeroDivisionError Occurs
  ↓
Exception Caught
  ↓
logging.exception()
  ↓
Create Log Record
  ↓
Include Traceback
  ↓
Display Error Message
```

Instead of terminating silently, Python records complete information about the exception.

Example

Example: An online banking application records exceptions whenever an invalid transaction causes an error.

Code

04_exception_logging.py

```
import logging

logging.basicConfig(
    level=logging.ERROR
)

try:
    result = 10 / 0

except Exception as e:
    logging.exception(
```

```
)  
    "An Exception Occurred"
```

Code Explanation

Step 1

```
import logging
```

Imports Python's built-in `logging` module.

Step 2

```
logging.basicConfig(  
    level=logging.ERROR  
)
```

Configures the logging system.

Only log messages with the **ERROR** level and above will be displayed.

Step 3

```
try:
```

The code inside the `try` block is executed.

If an exception occurs, Python immediately moves to the `except` block.

Step 4

```
result = 10 / 0
```

Attempts to divide 10 by 0.

Since division by zero is not allowed, Python raises a `ZeroDivisionError`.

Step 5

```
except Exception as e:
```

Catches the exception.

The variable `e` stores information about the exception.

Step 6

```
logging.exception(
    "An Exception Occurred"
)
```

Logs the exception.

Unlike `logging.error()`, `logging.exception()` automatically includes the complete traceback, making debugging much easier.

Output

```
ERROR:root:An Exception Occurred
Traceback (most recent call last):
  File "04_exception_logging.py", line 8, in <module>
    result = 10 / 0
ZeroDivisionError: division by zero
```



Understanding the Output

[illegible][illegible]



Error Description

The traceback shows:

- The file where the exception occurred.
- The exact line number.
- The type of exception.
- The reason for the exception.

This information helps developers quickly locate and fix the problem.

Benefits of Exception Logging

- Captures unexpected errors.
 - Records complete traceback information.
 - Makes debugging easier.
 - Improves application reliability.
 - Helps monitor production systems.
-

Key Takeaway

Exception Logging automatically records both the error message and the complete traceback whenever an exception occurs. This provides valuable information for debugging, monitoring, and maintaining reliable Python applications.

Logging Summary

Logging is the process of recording important events, warnings, errors, and exceptions that occur while a program is running.

Instead of relying on `print()` statements, Python's built-in `logging` module provides a structured and flexible way to monitor application behavior, track issues, and maintain execution history.

Throughout this module, we explored the following concepts:

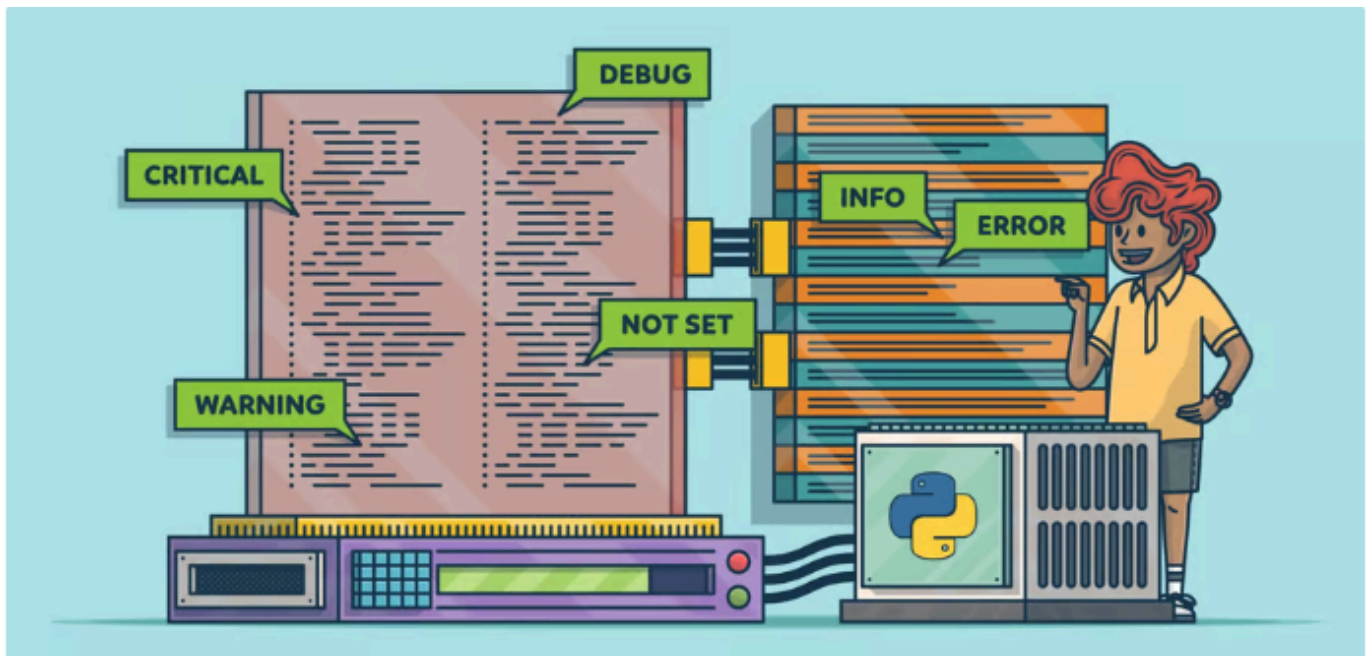
- Basic Logging
- File Logging
- Custom Formatter
- Exception Logging

By understanding these concepts, developers can effectively monitor applications, simplify debugging, record important events, and quickly identify problems. Logging is widely used in web applications, banking systems, cloud services, APIs, and enterprise software to improve reliability, maintenance, and production monitoring.

Configuration Management

- [Configuration Management Overview](#)
 - [Why Configuration Management?](#)
 - [Common Configuration Methods](#)
 - [Real-World Example](#)
 - [How Configuration Management Works](#)
 - [JSON Configuration](#)
 - [Read JSON Configuration](#)
 - [Environment Variables](#)
 - [ConfigParser Example](#)
 - [Summary](#)
-

Configuration Management Overview



Configuration Management is the practice of storing and managing application settings separately from the source code.

Instead of hardcoding values such as database credentials, API keys, file paths, and application settings inside the program, these values are stored in separate configuration files or environment variables.

This makes applications more secure, flexible, easier to maintain, and easier to deploy in different environments.

Understanding Configuration Management

Almost every application requires some settings to run.

Examples include:

- Database Host
- Database Username
- Database Password
- API Keys
- File Paths
- Application Name
- Debug Mode

Instead of writing these values directly inside the Python program, they are stored separately.

The application reads these settings whenever it starts.

The complete process is shown below.

```
Configuration File
  ↓
Application Starts
  ↓
Read Configuration
  ↓
Load Configuration Values
  ↓
Application Uses Settings
```

Let's understand each step.

Step 1: Configuration File

The application settings are stored in a separate configuration file.

Python commonly uses:

- JSON files
- INI files
- Environment Variables

Example:

```
{
    "app_name": "Student Management System",
    "debug": true
}
```

At this stage, the settings are simply stored outside the program.

Step 2: Application Starts

When the application starts, it first looks for the configuration file.

Instead of hardcoding values inside the program, it loads them from the configuration source.

For example:

```
Application Started
    ↓
Read config.json
```

Step 3: Read Configuration

Python reads the configuration using modules such as:

- `json`
- `configparser`
- `os`

For example:

```
import json
```

or

```
import configparser
```

or

```
import os
```

Each module reads a different type of configuration source.

Step 4: Load Configuration Values

After reading the configuration, Python converts the data into Python objects.

For example:

```
config["app_name"]
config["debug"]
```

Now the program can access these values whenever needed.

Step 5: Application Uses Settings

Finally, the application uses the configuration values during execution.

Example:

```
if config["debug"]:  
    print("Debug Mode Enabled")
```

Instead of changing the source code, developers only need to update the configuration file whenever settings change.

Why Use Configuration Management?

Configuration Management provides several advantages.

Improves Security

Sensitive information such as passwords and API keys is stored outside the source code.

Avoids Hardcoded Values

Application settings can be changed without modifying the program.

Simplifies Deployment

Different environments can use different configuration files.

Easy Maintenance

Updating application settings becomes much easier.

Supports Multiple Environments

Different configurations can be used for:

- Development
- Testing
- Production

Common Configuration Methods

Python supports several ways to manage application settings.

Method	Description
JSON Configuration	Stores configuration in JSON files
Environment Variables	Stores sensitive information securely
ConfigParser	Reads configuration from INI files

Real-World Example

Consider an **E-Commerce Application**.

Instead of writing settings directly in the code, developers store them separately.

For example:

- Database URL → JSON or INI file
- API Key → Environment Variable
- Debug Mode → Configuration File
- File Upload Path → Configuration File

When the application moves from **Development** to **Production**, developers only update the configuration values instead of changing the program code.

This makes the application secure, flexible, and easier to maintain.

Real-World Applications

Configuration Management is commonly used in:

- Web Applications
 - Banking Systems
 - E-Commerce Platforms
 - Cloud Applications
 - REST APIs
 - Microservices
 - Desktop Applications
 - Enterprise Software
-

Examples

Concept	File
JSON Configuration	01_config.json
Read JSON Config	02_json_config.py
Environment Variables	03_env_example.py
ConfigParser Example	04_configparser_example.py

1. JSON Configuration

Explanation



JSON (JavaScript Object Notation) is one of the most commonly used formats for storing application configuration.

A JSON configuration file stores application settings as **key-value pairs**, making the configuration easy to read, modify, and maintain.

Instead of hardcoding values such as database details, application names, or debug settings inside the source code, developers store them in a JSON file.

JSON is lightweight, human-readable, and supported by almost every programming language.

Why Use JSON Configuration?

JSON configuration is useful because it:

- Separates configuration from source code.
 - Makes applications easier to maintain.
 - Improves readability.
 - Allows configuration without modifying the program.
 - Is supported by most programming languages.
-

Syntax

```
{  
  "key": "value"  
}
```

Syntax Explanation

- `{ }` represents a JSON object.
 - `"key"` identifies the configuration setting.
 - `"value"` stores the corresponding value.
 - Keys and values are separated using a colon (`:`).
-

Code Flow

```
Create JSON File  
↓
```

```
Store Configuration Settings
    ↓
Save File
    ↓
Application Reads File
    ↓
Configuration Ready to Use
```

How JSON Configuration Works

```
Developer Creates config.json
    ↓
Application Starts
    ↓
Configuration File Available
    ↓
Python Reads File
    ↓
Configuration Values Loaded
```

The JSON file stores only the configuration values.

The application will read these values whenever it starts.

Example

Example: Store application information and database settings in a JSON configuration file.

Configuration File

01_config.json

```
{
  "app_name": "Student Management System",
  "debug": true,
  "database": {
    "host": "localhost",
    "port": 3306
  }
}
```

Understanding the JSON Structure

```
{
  "app_name": "Student Management System",
  "debug": true,
  "database": {
    "host": "localhost",
    "port": 3306
  }
}
```

app_name

Stores the application's name.

```
"app_name": "Student Management System"
```

debug

Specifies whether Debug Mode is enabled.

```
"debug": true
```

database

Groups all database-related settings.

```
"database": {
  ...
}
```

host

Stores the database server address.

```
"host": "localhost"
```

port

Stores the database port number.

```
"port": 3306
```

Benefits of JSON Configuration

- Easy to read.
- Easy to modify.
- Lightweight format.
- Language independent.
- Widely used in modern applications.

Key Takeaway

JSON Configuration provides a simple, structured, and readable way to store application settings separately from the source code, making applications more secure and easier to maintain.

2. Read JSON Configuration

Explanation

Python provides the built-in `json` module to read JSON configuration files.

The `json.load()` function reads the JSON file and converts it into a Python dictionary.

Once loaded, the application can access configuration values just like dictionary keys.

This allows applications to load settings dynamically without modifying the source code.

Why Read JSON Configuration?

Reading JSON files allows applications to:

- Load configuration dynamically.
- Avoid hardcoded values.
- Separate configuration from code.
- Simplify maintenance.
- Reuse the same application in different environments.

Syntax

```
import json
```

```
with open("config.json", "r") as file:  
    config = json.load(file)
```

Syntax Explanation

- `open()` opens the JSON file.
- `"r"` opens the file in read mode.
- `json.load()` converts the JSON data into a Python dictionary.
- `config` stores all configuration values.

Code Flow

```
Open JSON File  
↓  
Read File Contents  
↓  
Convert JSON into Dictionary  
↓  
Store Configuration  
↓  
Access Required Values
```

How Python Executes

When Python executes:

```
config = json.load(file)
```

the following steps occur:

```
Application Starts  
↓  
Open config.json  
↓  
Read JSON Data  
↓  
Convert to Dictionary  
↓  
Store in config Variable  
↓  
Access Required Values
```

Example

Example: Read application settings from a JSON configuration file.

Configuration File

01_config.json

```
{
  "app_name": "Student Management System",
  "debug": true,
  "database": {
    "host": "localhost",
    "port": 3306
  }
}
```

Code

02_json_config.py

```
import json

with open("01_config.json", "r") as file:
    config = json.load(file)

print("Application Name:",
      config["app_name"])

print("Debug Mode:",
      config["debug"])

print("Database Host:",
      config["database"]["host"])

print("Database Port:",
      config["database"]["port"])
```

Code Explanation

Step 1

```
import json
```

Imports Python's built-in `json` module.

Step 2

```
with open("01_config.json", "r") as file:
```

Opens the JSON configuration file in read mode.

Step 3

```
config = json.load(file)
```

Reads the JSON file and converts its contents into a Python dictionary.

Step 4

```
config["app_name"]
```

Retrieves the application name.

Step 5

```
config["debug"]
```

Retrieves the Debug Mode setting.

Step 6

```
config["database"]["host"]
```

Retrieves the database host.

Step 7

```
config["database"]["port"]
```

Retrieves the database port number.

Output

```
Application Name: Student Management System
Debug Mode: True
Database Host: localhost
Database Port: 3306
```

Understanding the Output

```
Application Name: Student Management System
|
| Value from config["app_name"]
|_____ Configuration Key

Debug Mode: True
|
| Value from config["debug"]
|_____ Configuration Key

Database Host: localhost
|
| Value from config["database"]["host"]
|_____ Configuration Key

Database Port: 3306
|
| Value from config["database"]["port"]
|_____ Configuration Key
```

Each value is retrieved from the Python dictionary created by `json.load()`.

Benefits of Reading JSON Files

- Dynamic configuration loading.
 - Easy maintenance.
 - Separation of configuration from code.
 - Improved flexibility.
 - Better application management.
-

Key Takeaway

The `json.load()` function reads a JSON configuration file, converts it into a Python dictionary, and allows applications to access configuration values dynamically without modifying the source code.

3. Environment Variables

Explanation

Environment Variables are values stored outside the application and operating system settings.

Instead of storing sensitive information such as passwords, API keys, and database credentials directly in the source code, these values are stored as environment variables.

The application reads these values whenever it starts, making the application more secure and easier to configure.

Why Use Environment Variables?

Environment Variables are useful because they:

- Improve application security.
 - Avoid hardcoding sensitive information.
 - Simplify deployment.
 - Support multiple environments.
 - Make applications easier to maintain.
-

Syntax

Windows (PowerShell)

```
$env:APP_ENV="Development"  
$env:API_KEY="12345ABC"
```

Linux / macOS

```
export APP_ENV="Development"  
export API_KEY="12345ABC"
```

Python

```
import os

value = os.getenv("APP_ENV")
```

Syntax Explanation

- `os.getenv()` reads an environment variable.
- `"APP_ENV"` is the variable name.
- The returned value is stored in a Python variable.

Code Flow

```
Set Environment Variable
  ↓
Application Starts
  ↓
Python Reads Variable
  ↓
Store Value
  ↓
Application Uses Value
```

How Python Executes

When Python executes:

```
app_env = os.getenv("APP_ENV")
```

the following steps occur:

```
Application Starts
  ↓
Read APP_ENV
  ↓
Retrieve Value
  ↓
Store in app_env
  ↓
Application Uses Value
```

If the environment variable does not exist, `None` is returned.

Example

Example: Read the application environment and API key from environment variables.

Setting Environment Variables

Windows

```
$env:APP_ENV="Development"
$env:API_KEY="12345ABC"
```

Linux / macOS

```
export APP_ENV="Development"
export API_KEY="12345ABC"
```

Code

03_env_example.py

```
import os

app_env = os.getenv("APP_ENV")
api_key = os.getenv("API_KEY")

print("Environment:", app_env)
print("API Key:", api_key)
```

Code Explanation

Step 1

```
import os
```

Imports Python's built-in `os` module.

Step 2

```
app_env = os.getenv("APP_ENV")
```

Reads the value stored in the `APP_ENV` environment variable.

Step 3

```
api_key = os.getenv("API_KEY")
```

Reads the API key from the operating system.

Step 4

```
print("Environment:", app_env)
```

Displays the application environment.

Step 5

```
print("API Key:", api_key)
```

Displays the API key.

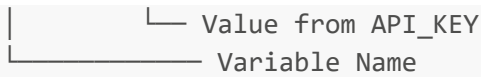
Output

```
Environment: Development
API Key: 12345ABC
```

Understanding the Output

```
Environment: Development
|           |
|           | Value from APP_ENV
|_____ Variable Name

API Key: 12345ABC
|           |
```



The values are retrieved from the operating system instead of being hardcoded in the program.

Benefits of Environment Variables

- Improves security.
 - Avoids hardcoded secrets.
 - Easy deployment.
 - Supports cloud platforms.
 - Simplifies configuration management.
-

Key Takeaway

Environment Variables provide a secure and flexible way to store sensitive information outside the application. Python reads these values during execution, making applications safer and easier to configure.

4. ConfigParser Example

Explanation

ConfigParser is a built-in Python module used to read and manage configuration files in **INI** format.

INI files organize application settings into sections and key-value pairs, making them simple to read and maintain.

Instead of hardcoding settings inside the source code, applications can store them in an INI file and load them whenever the program starts.

Why Use ConfigParser?

ConfigParser is useful because it:

- Organizes settings into sections.
 - Separates configuration from source code.
 - Simplifies maintenance.
 - Supports multiple configuration sections.
 - Is built into Python.
-

Syntax

```
import configparser

config = configparser.ConfigParser()

config.read("config.ini")
```

Syntax Explanation

- `ConfigParser()` creates a configuration object.
- `read()` loads the INI file.
- Configuration values are accessed using section names and keys.

Code Flow

```
Create INI File
    ↓
Application Starts
    ↓
Read Configuration File
    ↓
Load Configuration
    ↓
Access Values
```

How Python Executes

When Python executes:

```
config.read("config.ini")
```

the following steps occur:

```
Application Starts
    ↓
Open config.ini
    ↓
Read Sections
    ↓
Load Key-Value Pairs
    ↓
Store Configuration
```

↓
Application Uses Values

Example

Example: Read database settings from an INI configuration file.

Configuration File

config.ini

```
[DATABASE]
host = localhost
port = 3306
username = admin
password = admin123
```

Code

04_configparser_example.py

```
import configparser

config = configparser.ConfigParser()

config.read("config.ini")

print("Host:",
      config["DATABASE"]["host"])

print("Port:",
      config["DATABASE"]["port"])

print("Username:",
      config["DATABASE"]["username"])
```

Code Explanation

Step 1

```
import configparser
```

Imports Python's built-in `configparser` module.

Step 2

```
config = configparser.ConfigParser()
```

Creates a ConfigParser object.

Step 3

```
config.read("config.ini")
```

Reads the INI configuration file.

Step 4

```
config["DATABASE"]["host"]
```

Retrieves the database host.

Step 5

```
config["DATABASE"]["port"]
```

Retrieves the database port.

Step 6

```
config["DATABASE"]["username"]
```

Retrieves the database username.

Output

```
Host: localhost
Port: 3306
Username: admin
```

Understanding the Output

```
Host: localhost
|      |
|      | Value from host
|      |
|      | Configuration Key

Port: 3306
|      |
|      | Value from port
|      |
|      | Configuration Key

Username: admin
|      |
|      | Value from username
|      |
|      | Configuration Key
```

The values are read directly from the INI configuration file.

Benefits of ConfigParser

- Built into Python.
 - Easy to organize settings.
 - Supports multiple sections.
 - Easy to modify.
 - Suitable for small and medium-sized applications.
-

Key Takeaway

ConfigParser provides a simple and organized way to store application settings in INI files. It helps separate configuration from application logic, making programs easier to maintain and configure.

Configuration Summary

Configuration Management is the practice of storing and managing application settings separately from the source code.

Instead of hardcoding values such as database credentials, API keys, file paths, and application settings, developers store them in configuration files or environment variables.

Throughout this module, we explored the following concepts:

- JSON Configuration
- Read JSON Configuration
- Environment Variables
- ConfigParser Example

By understanding these concepts, developers can build applications that are more secure, flexible, and easier to maintain. Configuration Management is widely used in web applications, cloud services, enterprise software, APIs, and microservices to simplify deployment and support different environments such as Development, Testing, and Production.

Python Best Practices

- [Python Best Practices Overview](#)
 - [Why Follow Best Practices?](#)
 - [Key Best Practices](#)
 - [Real-World Example](#)
 - [How Best Practices Help](#)
 - [PEP 8 Example](#)
 - [Type Hints](#)
 - [Exception Handling](#)
 - [Docstrings](#)
 - [Unit Testing](#)
 - [Summary](#)
-

Python Best Practices

Python Best Practices Overview

Python Best Practices are a set of guidelines and standards that help developers write clean, readable, maintainable, and efficient code.

Following these practices improves code quality, reduces bugs, makes collaboration easier, and helps developers build professional software applications.

Python provides several standards and techniques such as **PEP 8**, **Type Hints**, **Exception Handling**, **Docstrings**, and **Unit Testing** to encourage writing high-quality code.

Understanding Python Best Practices

When beginners start learning Python, they usually focus on writing code that works.

For example:

```
print("Hello World")
```

The program runs correctly, so it seems fine.

However, as applications become larger, thousands of lines of code are written by multiple developers. If everyone writes code in a different style, the application becomes difficult to read, understand, debug, and maintain.

Python Best Practices provide a set of guidelines that help developers write clean, organized, and professional code.

The complete process is shown below.

```
Developer Writes Code
↓
Follow Python Standards
↓
Code Becomes Easy to Read
↓
Fewer Bugs and Errors
↓
Easy to Maintain and Update
↓
Reliable Software
```

Let's understand each step.

Step 1: Developer Writes Code

Every application begins with writing Python code.

For example:

```
name="Avinash"
print(name)
```

The program works correctly, but the code may not follow Python standards.

Different developers may write the same program in different ways.

Step 2: Follow Python Standards

Instead of writing code in any style, Python recommends following standard practices.

Some important best practices are:

- PEP 8 (Coding Style)
- Type Hints
- Exception Handling
- Docstrings
- Unit Testing

These standards make the code consistent and easier to understand.

Step 3: Code Becomes Easy to Read

After following Python standards, the code becomes cleaner.

For example:

```
user_name = "Avinash"  
  
print(user_name)
```

Compared to:

```
a="Avinash"  
print(a)
```

The first example is much easier to understand because the variable name clearly explains its purpose.

When code is readable, other developers can understand it quickly.

Step 4: Fewer Bugs and Errors

Best practices help prevent common programming mistakes.

For example:

- Exception Handling prevents applications from crashing.
- Type Hints reduce incorrect data usage.
- Unit Tests detect bugs before deployment.

This improves the overall quality of the application.

Step 5: Easy to Maintain and Update

Applications are rarely written once.

Developers continuously:

- Add new features.
- Fix bugs.
- Improve performance.
- Update existing functionality.

When code follows best practices, these changes become much easier because the code is organized and well documented.

Step 6: Reliable Software

Finally, applications become more reliable.

Developers can easily:

- Understand the code.
- Fix problems.
- Add new features.
- Work together as a team.

This is why almost every professional Python project follows best practices.

Why Follow Python Best Practices?

Following Python Best Practices provides several advantages.

Improves Code Readability

Well-formatted and organized code is easier to understand.

Simplifies Maintenance

Developers can update and modify applications more easily.

Reduces Bugs

Using proper coding standards helps prevent programming mistakes.

Improves Team Collaboration

When everyone follows the same coding style, team members can easily understand each other's code.

Follows Industry Standards

Most professional Python projects follow these standards to produce high-quality software.

Real-World Example

Consider a **Banking Application**.

A banking application performs many operations such as customer registration, money transfers, balance checks, and transaction processing. To keep the application reliable and easy to maintain, developers follow Python Best Practices.

- **PEP 8** keeps the source code clean and consistently formatted.
- **Type Hints** make it clear what type of data a function expects and returns.
- **Exception Handling** prevents the application from crashing when unexpected errors occur.
- **Docstrings** explain the purpose of functions, classes, and modules so other developers can easily understand the code.
- **Unit Testing** verifies that important features, such as money transfers and balance calculations, work correctly before deployment.

By following these best practices, developers can build applications that are easier to read, maintain, test, and scale.

Examples

Concept	File
PEP 8 Example	01_pep8_example.py
Type Hints	02_type_hints.py
Exception Handling	03_exception_handling.py
Docstrings	04_docstrings.py
Unit Testing	05_unit_test_example.py

Understanding PEP 8

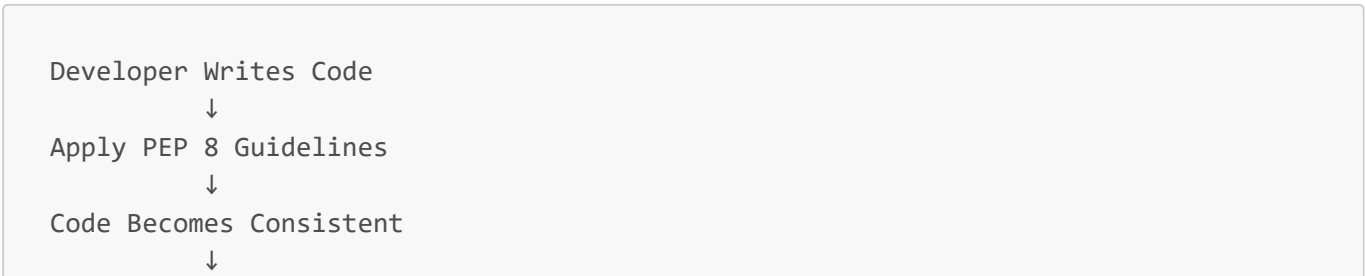
PEP 8 (Python Enhancement Proposal 8) is the official style guide for writing Python code.

It provides a set of coding conventions that help developers write code in a clean, readable, and consistent way.

Imagine that five developers are working on the same Python project. If each developer writes code using a different style, the project becomes difficult to read, understand, and maintain.

PEP 8 solves this problem by providing a common set of rules that everyone follows.

The complete process is shown below.



Easy to Read
↓
Easy to Maintain
↓
Professional Python Code

Let's understand each step.

Step 1: Developer Writes Code

A developer writes a Python program.

Example:

```
a=10  
b=20  
print(a+b)
```

Output

```
30
```

The program works correctly, but it does not follow PEP 8 coding standards.

Step 2: Apply PEP 8 Guidelines

The developer formats the same program according to PEP 8 recommendations.

Example:

```
a = 10  
b = 20  
  
print(a + b)
```

Output

```
30
```

The output is exactly the same, but the code is much cleaner and easier to read.

Step 3: Code Becomes Consistent

When all developers follow PEP 8, every Python file follows the same coding style.

This makes the project more organized and easier for everyone to understand.

Step 4: Easy to Read

PEP 8 improves readability by using:

- Meaningful variable names
- Proper indentation
- Spaces around operators
- Consistent formatting

As a result, developers can quickly understand the purpose of the code.

Step 5: Easy to Maintain

Software projects are continuously updated with new features and bug fixes.

When the code follows PEP 8, developers can easily modify and maintain the application because the code is well organized and consistently formatted.

Step 6: Professional Python Code

Following PEP 8 helps developers write clean, readable, maintainable, and professional Python code.

It is the coding standard followed in most professional Python projects and open-source libraries.

Why Follow PEP 8?

Following PEP 8 provides several advantages.

- Improves code readability.
 - Makes code easier to maintain.
 - Encourages team collaboration.
 - Reduces coding errors.
 - Follows Python's official coding standards.
 - Makes projects look clean and professional.
-

Basic PEP 8 Style

PEP 8 does not introduce new Python syntax. Instead, it provides rules for writing Python code in a clean and consistent style.

Some of the most important style rules include:

- Use meaningful names.

- Use **snake_case** for variables and functions.
 - Use **PascalCase** for class names.
 - Use four spaces for indentation.
 - Add spaces around operators.
 - Separate functions with blank lines.
 - Keep code simple and readable.
-

Important PEP 8 Guidelines

1. Use Meaningful Names

Choose names that clearly describe their purpose.

Good

```
student_name = "Avinash"  
total_marks = 450
```

Bad

```
a = "Avinash"  
x = 450
```

2. Use Snake Case for Variables and Functions

PEP 8 recommends using **snake_case** for variable names and function names.

In **snake_case**, all letters are written in lowercase, and multiple words are separated using an underscore (_).

This naming style makes code easier to read and is the standard convention used in Python.

Good

```
student_name = "Avinash"  
  
def calculate_total():  
    pass
```

Bad

```
studentName = "Avinash"
```

```
def CalculateTotal():  
    pass
```

3. Use Pascal Case for Class Names

Class names should use **PascalCase**.

Good

```
class StudentDetails:  
    pass
```

Bad

```
class student_details:  
    pass
```

4. Use Four Spaces for Indentation

Always indent code using four spaces.

Good

```
if True:  
    print("Hello")
```

Bad

```
if True:  
print("Hello")
```

5. Add Spaces Around Operators

Spaces improve readability.

Good

```
total = marks + bonus
```

Bad

```
total=marks+bonus
```

6. Separate Functions with Blank Lines

Leave two blank lines between top-level functions.

Good

```
def add():  
    pass  
  
def subtract():  
    pass
```

7. Keep Code Readable

Write code that is simple, clean, and easy to understand.

Avoid unnecessary complexity and follow a consistent coding style throughout your project.

Understanding Type Hints

Python is a **dynamically typed language**, which means you do not need to specify the data type of variables or function parameters.

For example:

```
name = "Avinash"  
age = 21
```

Python automatically understands that:

- `name` is a string.
- `age` is an integer.

Although this makes Python easy to write, it can make large programs difficult to understand because developers may not know what type of data a function expects.

Type Hints solve this problem by allowing developers to specify the expected data types of variables, function parameters, and return values.

The complete process is shown below.

```
Write Python Function
  ↓
Add Type Hints
  ↓
Specify Expected Data Types
  ↓
Improve Code Readability
  ↓
Reduce Programming Mistakes
  ↓
Maintain Code Easily
```

Let's understand each step.

Step 1: Write a Function

A developer creates a normal Python function.

```
def greet(name):
    print(name)
```

The function works correctly, but it does not indicate what type of value `name` should be.

Step 2: Add Type Hints

The developer adds type hints.

```
def greet(name: str):
    print(name)
```

Now it is clear that the function expects a **string**.

Step 3: Specify Expected Data Types

Type Hints can be added to:

- Variables
- Function Parameters
- Return Values

This makes the program easier to understand.

Step 4: Improve Code Readability

Developers can immediately understand what type of data a function expects and returns.

Step 5: Reduce Programming Mistakes

Modern code editors and type checkers can warn developers when the wrong data type is used.

Step 6: Maintain Code Easily

When applications grow larger, Type Hints make the code easier to understand, update, and maintain.

Understanding Exception Handling

When a Python program is running, unexpected errors may occur.

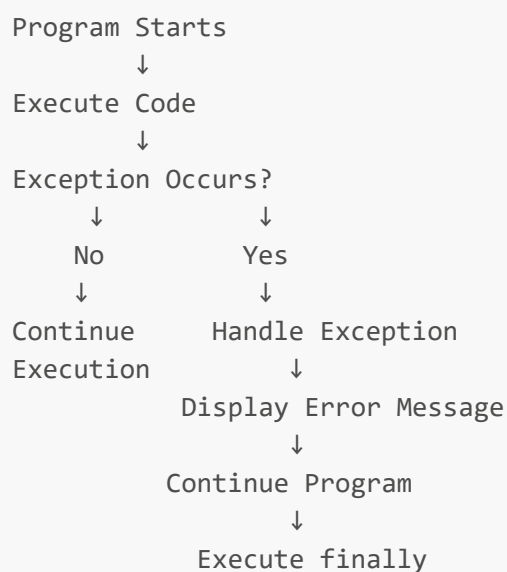
For example:

- A user enters text instead of a number.
- A file does not exist.
- A program tries to divide a number by zero.
- A network connection fails.

If these errors are not handled, the program stops immediately and displays an error message.

Exception Handling allows the program to detect these errors and respond appropriately instead of crashing.

The complete process is shown below.



Let's understand each step.

Step 1: Program Starts

Python begins executing the program.

Step 2: Execute Code

The statements inside the `try` block are executed.

```
try:  
    result = 100 / number
```

Python checks whether any exception occurs.

Step 3: Exception Occurs?

If no exception occurs:

```
Continue Normal Execution
```

If an exception occurs:

```
Move to except Block
```

Step 4: Handle Exception

Python looks for a matching `except` block.

For example:

```
except ZeroDivisionError:
```

or

```
except ValueError:
```

The appropriate error message is displayed.

Step 5: Continue Program

Instead of terminating the application, Python continues executing the remaining statements.

Step 6: Execute finally

The **finally** block always executes.

Whether an exception occurs or not, the code inside **finally** runs before the program ends.

4. Docstrings

Explanation

Docstrings (Documentation Strings) are special strings used to describe the purpose, functionality, parameters, and return values of a module, class, or function.

They are written using triple quotes (`""" """`) and are placed immediately below the definition of a module, class, or function.

Unlike comments, docstrings become part of the program and can be accessed using Python's built-in **help()** function or the `__doc__` attribute.

Docstrings make code easier to understand, maintain, and document, making them one of Python's best practices.

Understanding Docstrings

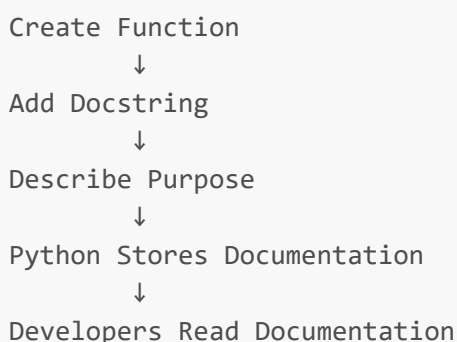
When developers write Python programs, they create many functions, classes, and modules.

As the project grows, it becomes difficult to remember what each function does or what kind of input it expects.

Docstrings solve this problem by providing built-in documentation directly inside the code.

Instead of reading the entire implementation, developers can simply read the docstring to understand the purpose of the code.

The complete process is shown below.



↓
Maintain Code Easily

Let's understand each step.

Step 1: Create a Function

The developer creates a function.

Example:

```
def add(a, b):  
    return a + b
```

The function works correctly, but there is no explanation of what it does.

Step 2: Add a Docstring

A docstring is added immediately below the function definition.

```
def add(a, b):  
    """Add two numbers."""
```

Now anyone reading the code immediately understands the function's purpose.

Step 3: Describe the Function

A docstring can describe:

- The purpose of the function
- The parameters
- The return value
- Exceptions
- Usage examples

This makes the function self-documenting.

Step 4: Python Stores the Documentation

Python automatically stores the docstring.

Developers do not need to write any extra code to save it.

Step 5: Read the Documentation

Developers can view the documentation using:

```
help(add)
```

or

```
print(add.__doc__)
```

This displays the documentation without opening the source code.

Step 6: Maintain Code Easily

Since every important function is documented, developers can quickly understand, update, and maintain the application.

Why Use Docstrings?

Docstrings provide several advantages.

Improves Code Readability

Docstrings explain the purpose of functions, classes, and modules, making the code easier to understand.

Provides Built-in Documentation

Python automatically stores docstrings, allowing developers to access them using `help()` or `__doc__`.

Simplifies Maintenance

Well-documented code is easier to update because developers can quickly understand how it works.

Helps Team Collaboration

When multiple developers work on the same project, docstrings make it easier for everyone to understand the code.

Supports Documentation Tools

Many documentation tools automatically generate project documentation from Python docstrings.

Syntax

Function Docstring

```
def function_name():  
    """  
    Function description.  
    """
```

Class Docstring

```
class Student:  
    """  
    Class description.  
    """
```

Module Docstring

```
"""  
Module description.  
"""
```

Syntax Explanation

- Triple quotes (""" """) are used to create docstrings.
- A docstring is written immediately below the definition of a function, class, or module.
- Python automatically stores the docstring.
- Developers can view it using:
 - `help()`
 - `__doc__`

Types of Docstrings

Python supports different styles of writing docstrings.

The most common types are:

1. Triple-Quoted Strings
2. One-line Docstrings
3. Multi-line Docstrings
4. Google Style Docstrings
5. NumPy Style Docstrings

1. Triple-Quoted Strings

Triple-quoted strings are the standard way to create docstrings in Python.

A docstring is written using triple double quotes (""" """) immediately below the definition of a function, class, or module.

Although Python also supports triple single quotes (''' '''), **PEP 257 recommends using triple double quotes** because they improve consistency across Python projects.

Example: Triple Single Quotes

```
def my_func():
    '''This is a docstring using triple single quotes.'''
    return None

print("Using __doc__:")
print(my_func.__doc__)

print("\nUsing help():")
help(my_func)
```

Output

```
Using __doc__:
This is a docstring using triple single quotes.

Using help():
Help on function my_func in module __main__:

my_func()
    This is a docstring using triple single quotes.
```

python output

Example: Triple Double Quotes (Recommended)

```
def my_func_preferred():
    """This is a docstring using triple double quotes."""
    return None

print(my_func_preferred.__doc__)
```

Output

```
This is a docstring using triple double quotes.
```

python output

Why Use Triple Double Quotes?

- Recommended by PEP 257.
- Improves consistency across projects.
- Easier to read.
- Widely used in professional Python applications.

2. One-line Docstrings

A One-line Docstring is used when a function, class, or module performs a simple task that can be explained in a single sentence.

The opening and closing triple quotes are written on the same line.

Example

```
def power(a, b):  
    """Return a raised to the power of b."""  
    return a ** b  
  
print(power.__doc__)
```

Output

```
Return a raised to the power of b.
```

python output

When to Use One-line Docstrings?

Use one-line docstrings when:

- The function performs a simple task.
- A single sentence is enough to explain its purpose.
- No parameter or return value explanation is required.

3. Multi-line Docstrings

Multi-line Docstrings are used when additional information is needed.

They usually contain:

- A short summary.
- A description.
- Parameters.
- Return values.
- Additional notes if required.

Example

```
def add(a, b):  
    """  
    Add two numbers.  
  
    Parameters:  
        a (int): First number.  
        b (int): Second number.  
  
    Returns:  
        int: Sum of a and b.  
    """  
    return a + b  
  
print(add(3, 7))
```

Output

```
10
```

python output

When to Use Multi-line Docstrings?

Use multi-line docstrings when:

- The function has parameters.
- The function returns a value.
- Additional explanation is needed.
- The function performs multiple operations.

4. Google Style Docstrings

Google Style Docstrings organize documentation into clearly labeled sections such as **Args** and **Returns**.

This format is easy to read and is commonly used in professional software projects.

Example

```
def multiply(a, b):  
    """  
    Multiply two numbers.  
  
    Args:  
        a (int): First number.  
        b (int): Second number.  
  
    Returns:  
        int: Product of a and b.  
    """  
    return a * b  
  
print(multiply(3, 5))
```

Output

15

python output

Why Use Google Style Docstrings?

- Easy to read.
- Organized format.
- Popular in professional Python projects.
- Supported by many documentation tools.

5. NumPy Style Docstrings

NumPy Style Docstrings are commonly used in Data Science, Machine Learning, and Scientific Computing projects.

Instead of labels like **Args**, NumPy uses section headings such as **Parameters** and **Returns**.

Example

```
def divide(a, b):  
    """  
    Divide two numbers.
```

```

Parameters
-----
a : float
    Dividend.
b : float
    Divisor.

Returns
-----
float
    Quotient of division.
"""
if b == 0:
    raise ValueError("Division by zero not allowed.")

return a / b

print(divide(6, 2))

```

Output

```
3.0
```

python best practices

Why Use NumPy Style Docstrings?

- Standard in scientific computing.
- Easy to organize large documentation.
- Commonly used in NumPy, Pandas, and SciPy.
- Improves documentation readability.

Additional Examples

The following files demonstrate different styles of writing docstrings.

File	Description
04.1_triple_single_docstring.py	Triple single-quoted docstring
04.2_triple_double_docstring.py	Triple double-quoted docstring
04.3_one_line_docstring.py	One-line docstring
04.4_multi_line_docstring.py	Multi-line docstring
04.5_google_style_docstring.py	Google style docstring

File	Description
04.6_numpy_style_docstring.py	NumPy style docstring

5. Unit Testing

Explanation



Unit Testing is the process of testing individual units or components of a program to ensure they work correctly.

A **unit** can be a function, method, or class.

Instead of manually checking whether a program works every time it is modified, developers write automated test cases that verify the expected behavior.

Python provides the built-in `unittest` module to create and run test cases.

Unit Testing helps developers detect bugs early, improve software quality, and ensure that future code changes do not break existing functionality.

Understanding Unit Testing

Whenever developers write a program, they need to verify that it works correctly.

For small programs, developers can manually check the output.

However, as applications become larger, manually testing every function becomes difficult and time-consuming.

Unit Testing solves this problem by automatically testing each function or component.

The complete process is shown below.

```
Write Function
    ↓
Create Test Cases
    ↓
Run Tests
    ↓
Compare Expected Result
    ↓
Test Passes or Fails
    ↓
Fix Errors if Needed
```

Let's understand each step.

Step 1: Write a Function

The developer creates a function.

Example:

```
def add(a, b):  
    return a + b
```

Step 2: Create Test Cases

A test case is written to verify whether the function produces the correct result.

Step 3: Run the Tests

Python executes all the test cases.

Step 4: Compare Results

Python compares:

- Expected Result
- Actual Result

If both match, the test passes.

Step 5: Pass or Fail

If the expected result matches the actual result, the test passes.

Otherwise, Python reports a failure.

Step 6: Fix Errors

If any test fails, the developer fixes the program and runs the tests again.

Why Use Unit Testing?

Unit Testing provides several advantages.

Detects Bugs Early

Problems are identified before the application is released.

Improves Code Reliability

Developers can verify that every function works correctly.

Simplifies Maintenance

Changes can be made confidently because tests verify existing functionality.

Encourages Modular Design

Functions become smaller and easier to test.

Prevents Regression

Previously working code continues to work after updates.

Positive vs Negative Test Cases

When testing software, developers create different kinds of test cases.

Positive Test Cases

Positive test cases verify that the program works correctly when valid input is provided.

Example:

```
add(20, 10)
```

Expected Output:

```
30
```

Negative Test Cases

Negative test cases verify that the program handles invalid input properly without crashing.

Example:

```
add("Ram", 10)
```

Instead of crashing unexpectedly, the application should display an appropriate error or handle the situation gracefully.

Testing both positive and negative test cases helps developers build reliable applications.

Syntax

```
import unittest

class TestExample(unittest.TestCase):

    def test_example(self):
        self.assertEqual(...)
```

Syntax Explanation

- `import unittest` imports Python's built-in testing framework.
- `TestCase` is the base class for writing test cases.
- Every method whose name starts with `test_` is treated as a test case.
- Assertions compare the expected result with the actual result.

Unit Testing Process

```
Write Function
  ↓
Create Test Case
  ↓
Run Test
  ↓
Compare Result
  ↓
Pass or Fail
```

How Python Executes

When Python executes:

```
unittest.main()
```

the following steps occur.

```
Program Starts
  ↓
Find Test Classes
  ↓
Find test_ Methods
  ↓
```

Run Test Cases



Compare Expected and Actual Results



Generate Test Report

Example

Example: Test a function that adds two numbers.

Code

05_unit_test_example.py

```
import unittest

def add(a, b):
    return a + b

class TestMathOperations(unittest.TestCase):

    def test_add(self):
        self.assertEqual(
            add(10, 5),
            15
        )

if __name__ == "__main__":
    unittest.main()
```

Code Explanation

Step 1

```
import unittest
```

Imports Python's built-in `unittest` module, which provides tools for creating and running test cases.

Step 2

```
def add(a, b):  
    return a + b
```

Defines a function named `add()` that returns the sum of two numbers.

This is the function we want to test.

Step 3

```
class TestMathOperations(unittest.TestCase):
```

Creates a test class.

Every test class should inherit from `unittest.TestCase`.

This allows Python to recognize and execute the test cases inside the class.

Step 4

```
def test_add(self):
```

Defines a test case.

Python automatically executes every method whose name starts with `test_`.

Step 5

```
self.assertEqual(  
    add(10, 5),  
    15  
)
```

Calls the `add()` function.

Python compares:

- **Actual Result:** `add(10, 5)`
- **Expected Result:** `15`

If both values are equal, the test passes.

Otherwise, the test fails.

Step 6

```
if __name__ == "__main__":  
    unittest.main()
```

Starts the Unit Testing framework.

Python searches for all test classes and executes every test method automatically.

Output

```
.  
-----  
Ran 1 test in 0.001s  
  
OK
```

 Unit Test Output

Understanding the Output

```
.  
|  
└─ Test Passed Successfully  
  
-----  
Ran 1 test  
|  
└─ One Test Case Executed  
  
OK  
|  
└─ All Test Cases Passed
```

The output indicates that:

- One test case was executed.
 - The expected result matched the actual result.
 - No errors or failures occurred.
-

Common Assertions

Assertions are methods used to verify whether the expected result matches the actual result.

Assertion	Description
<code>assertEqual(a, b)</code>	Checks whether two values are equal
<code>assertNotEqual(a, b)</code>	Checks whether two values are different
<code>assertTrue(x)</code>	Checks whether the value is <code>True</code>
<code>assertFalse(x)</code>	Checks whether the value is <code>False</code>
<code>assertIsNone(x)</code>	Checks whether the value is <code>None</code>
<code>assertIsNotNone(x)</code>	Checks whether the value is not <code>None</code>
<code>assertIn(a, b)</code>	Checks whether an item exists in a collection
<code>assertNotIn(a, b)</code>	Checks whether an item does not exist in a collection
<code>assertRaises()</code>	Checks whether a specific exception is raised

Benefits of Unit Testing

- Detects bugs early.
- Improves software quality.
- Simplifies debugging.
- Supports continuous integration.
- Reduces maintenance effort.
- Makes refactoring safer.
- Increases confidence when modifying code.

Key Takeaway

Unit Testing verifies that individual functions and components behave as expected.

By writing automated test cases, developers can quickly identify bugs, confidently modify existing code, and build reliable, maintainable, and high-quality Python applications.

Python Best Practices Summary

Python Best Practices are a collection of guidelines that help developers write clean, readable, maintainable, and professional Python code.

Instead of focusing only on making programs work, developers follow best practices to improve code quality, reduce bugs, simplify maintenance, and make applications easier to understand.

Throughout this module, we explored the following concepts:

- PEP 8 Example

- Type Hints
- Exception Handling
- Docstrings
- Unit Testing

By applying these best practices, developers can build applications that are easier to read, test, debug, maintain, and scale. These practices are widely used in professional Python projects, web applications, APIs, automation scripts, machine learning projects, and enterprise software.

Conclusion

Python is a powerful, versatile, and widely used programming language that supports everything from simple scripting to large-scale software development.

Throughout this repository, we explored both fundamental and advanced Python concepts, including Object-Oriented Programming, Data Structures, Decorators, Generators, Async Programming, Logging, Configuration Management, and Python Best Practices.

These concepts help developers write clean, efficient, maintainable, and scalable applications while following industry-standard development practices.

By understanding and applying these concepts, developers can build robust software solutions, improve code quality, and prepare for real-world software development challenges.

Continuous practice and hands-on implementation are the keys to mastering Python and becoming a proficient developer.

References

Official Python Documentation

- Python Documentation — <https://docs.python.org/3/>
- Python Tutorial — <https://docs.python.org/3/tutorial/>
- Python Standard Library — <https://docs.python.org/3/library/>

Module-Specific Documentation

- asyncio — <https://docs.python.org/3/library/asyncio.html>
- logging — <https://docs.python.org/3/library/logging.html>
- unittest — <https://docs.python.org/3/library/unittest.html>
- json — <https://docs.python.org/3/library/json.html>
- configparser — <https://docs.python.org/3/library/configparser.html>
- abc (Abstract Base Classes) — <https://docs.python.org/3/library/abc.html>
- collections (deque) — <https://docs.python.org/3/library/collections.html>
- heapq (Priority Queue) — <https://docs.python.org/3/library/heapq.html>

Python Enhancement Proposals (PEPs)

- PEP 8 – Style Guide for Python Code — <https://peps.python.org/pep-0008/>
- PEP 257 – Docstring Conventions — <https://peps.python.org/pep-0257/>
- PEP 484 – Type Hints — <https://peps.python.org/pep-0484/>

Learning Resources

- Real Python — <https://realpython.com/>
- GeeksforGeeks Python — <https://www.geeksforgeeks.org/python-programming-language/>

Articles and Blogs

- Medium Python Articles — <https://medium.com/tag/python>

This documentation was prepared using knowledge from the official Python documentation, Python Enhancement Proposals (PEPs), Real Python, GeeksforGeeks, Medium articles, and hands-on Python programming experience.
